

Notas de Aula

Prof. Walbert Cunha Monteiro

09.0.2026

Capítulo 1	A Linguagem de Programação JAVA	7
1.1	Componentes Principais	7
1.2	Características	8
1.3	Estrutura de um Programa em Java	9
Capítulo 2	Comentários	11
2.1	Comentário de Linha Única	11
2.2	Comentário de Múltiplas Linhas	11
2.3	Comentário de Documentação (Doc Comment - Javadoc)	11
2.4	Diferença entre comentários comuns e Javadoc	14
Capítulo 3	Variáveis	15
3.1	Tipos Primitivos	15
3.2	Tipos por Referência (Não Primitivos)	16
3.3	Conversão de Tipos (Casting)	18
3.3.1	Casting Implícito	18
3.4	Casting Explícito	18
3.4.1	Conversão de Strings para Tipos Primitivos	19
3.4.2	Conversão de Tipos Primitivos para String	20
3.5	Exercícios Resolvidos	21
Capítulo 4	Operadores	23
4.1	Operadores Aritméticos	23
4.2	Operadores de Atribuição	24
4.3	Operadores Relacionais ou de Comparação	25
4.4	Operadores Unários	25
4.5	Operadores Lógicos	27
4.5.1	Tabelas Verdade	28
4.6	Precedência de Operadores	30
4.7	Exercícios Resolvidos	30
Capítulo 5	Estruturas Condicionais	33
5.1	If (Simples)	33
5.2	If ... else (Composta)	33
5.3	If ... else if ... else (Encadeado)	34
5.4	Switch-case	34
5.4.1	Tipos de Dados Permitidos	35
5.4.2	Tipos Não Permitidos	37
5.5	Exercícios Resolvidos	38

Capítulo 6 Estruturas de Repetição	41
6.1 For	41
6.2 While	42
6.3 do-while.....	44
6.4 Break	45
6.5 Continue	46
Capítulo 7 Classes e Objetos	49
7.1 Estrutura básica de uma classe.....	49
7.2 Criando Classes	50
7.3 Criando Objetos	50
7.4 Modificadores de Acesso - Atributos e Métodos	52
7.4.1 Modificador public	52
7.4.2 Modificador private	52
7.4.3 Modificador protected	53
7.4.4 Modificador default	53
7.5 Modificadores de Acesso em Classes.....	54
7.5.1 Classe public.....	55
7.5.2 Classe default.....	55
7.6 Exercícios Resolvidos	56
Capítulo 8 Métodos	57
8.1 Métodos sem Retorno.....	58
8.1.1 Métodos sem retorno e com parâmetros	58
8.1.2 Métodos sem retorno e sem parâmetros	59
8.2 Métodos com Retorno de Valores.....	59
8.2.1 Métodos com retorno e com parâmetros	60
8.2.2 Métodos com retorno e sem parâmetros	60
8.3 Exercícios Práticos.....	61
Capítulo 9 Construtores	65
9.1 A Palavra-chave this	66
9.2 Construtor com Parâmetros	66
9.3 Sobrecarga de Construtores	67
9.4 Chamando um Construtor a partir de Outro	68
9.5 Construtores e Herança	68
Capítulo 10 Sobrecarga	71
10. Exercícios Resolvidos	72
Capítulo 11 Herança	73
11.1 Sintaxe da Herança	73
11.2 Vantagens da Herança	74
11.3 Herança e a palavra-chave final	76

Capítulo 12 Polimorfismo	79
12.1 Vantagens do Polimorfismo	80
12.2 Anotação @Override	82
Capítulo 13 Classes Abstratas	83
13.1 Métodos Abstratos	84
13.2 Classe Abstrata versus Classe Comum	86
Capítulo 14 Interfaces	87
14.1 Utilizando Objetos de Classes que Implementam Interfaces	87
14.2 Implementando Múltiplas Interfaces	88
14.3 Interfaces e Polimorfismo	89
14.4 Métodos Default	90
14.5 Métodos Estáticos em Interfaces	90
14.6 Diferenças entre Herança e Interface	90
Capítulo 15 Array Unidimensional	93
15.1 Criação	93
15.2 Tipos	94
15.3 Acesso	95
15.4 Pesquisa	96
15.5 Percorrer com for-each	96
15.6 Alteração	97
15.7 Remoção (limitações)	97
15.8 Exercícios Resolvidos	98
Capítulo 16 Arrays Multidimensionais	99
16.1 Criando um Array Bidimensional	99
16.2 Acessando Elementos de um Array Bidimensional	101
16.3 Removendo Elementos de um Array Bidimensional	101
16.4 Alterando o Valor de um Elemento	102
16.5 Percorrendo um Array Multidimensional	103
16.6 Pesquisando Elementos em um Array Bidimensional	105
Capítulo 17 ArrayList	107
17.1 Métodos	108
Capítulo 18 HashSet	113
18.1 Características	113
18.2 Métodos	113
Capítulo 19 HashMap	115
19.1 Características	115
19.2 Métodos	115

Capítulo 20 Fila	117
20.1 Características	117
20.2 Métodos	118
20.3 Exemplo	118
Capítulo 21 Stack	119
21.1 Características	119
21.2 Métodos	119
Capítulo 22 Exercícios	121

Capítulo 1 A Linguagem de Programação JAVA

Foi concebida em 1995 por engenheiros da Sun Microsystems. Atualmente, a Oracle Corporation é a proprietária desta tecnologia e a principal mantenedora da plataforma Java. Ao longo dos anos, a linguagem evoluiu para uma plataforma que oferece todo o ambiente necessário para desenvolver e executar aplicações, ganhando destaque como uma importante tecnologia na área da computação, atendendo a diferentes níveis de usuários e operando em diversos dispositivos.

Em função do grande sucesso e aceitação que a linguagem alcançou, foram desenvolvidos diferentes frameworks para facilitar o processo de desenvolvimento de software. Um framework é um conjunto de ferramentas, bibliotecas e convenções que oferece uma estrutura base para o desenvolvimento rápido e padronizado de software, são exemplos de frameworks: Spring, Hibernate, Vaadin, Struts.

1.1 Componentes Principais

Para entender como a linguagem Java funciona, é importante conhecer alguns componentes. Cada um tem uma função específica no processo de criação e execução de programas.

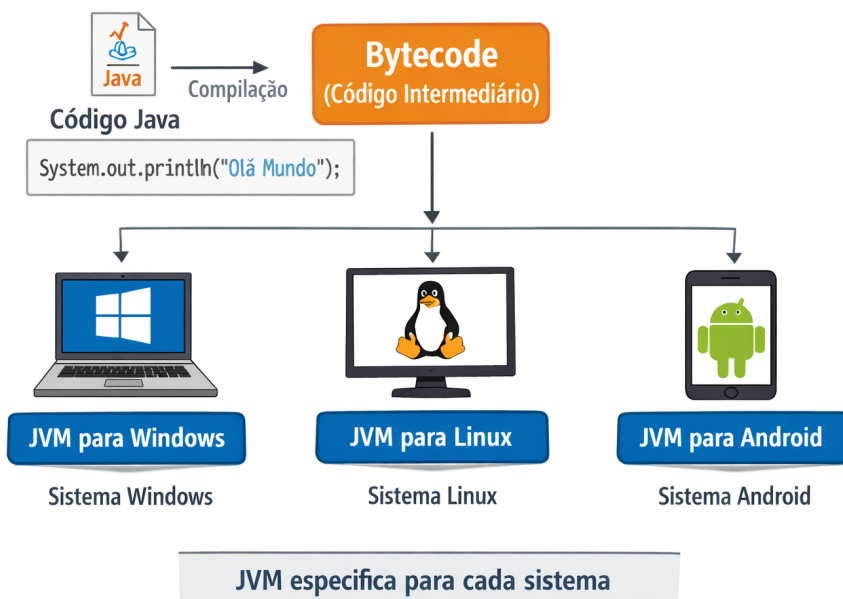
- ❁ **JVM (Java Virtual Machine):** A JVM é a Máquina Virtual Java. Ela é responsável por executar os programas Java. Em vez de rodar diretamente no sistema operacional, o programa é executado dentro da JVM. Isso permite que o mesmo código funcione em diferentes computadores.
- ❁ **JRE (Java Runtime Environment):** A JRE é o ambiente necessário para executar aplicações Java. Ela inclui a JVM e outras bibliotecas essenciais. Se você apenas deseja rodar programas Java, a JRE já é suficiente.
- ❁ **JDK (Java Development Kit):** A JDK é o conjunto completo de ferramentas para desenvolvimento em Java. Ela inclui a JRE e também ferramentas para escrever, compilar e testar programas. Quem deseja programar em Java precisa instalar o JDK.

O Java possui diferentes edições, cada uma voltada para um tipo de aplicação:

- ❁ **Java SE (Standard Edition):** É a base da linguagem. Usada para criar programas simples e aplicações desktop.
- ❁ **Java EE (Enterprise Edition):** Voltada para aplicações maiores, como sistemas web e corporativos.
- ❁ **Java ME (Micro Edition):** Utilizada em dispositivos com recursos limitados, como sistemas embarcados.

Graças à Java Virtual Machine (JVM), que interpreta o código compilado (bytecode), um programa Java pode ser executado em diferentes sistemas operacionais sem precisar ser modificado. Esse princípio é conhecido pela expressão **“Escreva uma vez, execute em qualquer lugar”** (*Write once, run anywhere* – WORA), e representa uma vantagem da linguagem: a portabilidade. A Figura 1.1 ilustra esse processo.

Figura 1.1. O código Java ao ser compilado é transformado em bytecode e executado pela JVM, permitindo que o mesmo programa funcione em diferentes sistemas operacionais.



1.2 Características

A linguagem Java possui ainda várias características que facilitam o desenvolvimento de programas e permitem seu uso em diferentes tipos de aplicações.

- ❁ **Independência de plataforma:** O código Java é compilado em *bytecode* e executado pela JVM. Isso permite que o mesmo programa funcione em diferentes sistemas operacionais, como Windows, Linux e macOS.
- ❁ **Orientação a objetos:** O código é organizado em classes e objetos que interagem entre si. Isso torna o programa mais organizado, reutilizável e fácil de manter.
- ❁ **Gerenciamento automático de memória:** O Java utiliza um mecanismo chamado *Garbage Collector*, que cuida automaticamente da liberação de memória, evitando erros comuns.
- ❁ **Segurança:** Possui mecanismos internos, como verificação de *bytecode* e controle de execução, que ajudam a proteger o sistema contra falhas e códigos maliciosos.
- ❁ **Desempenho:** Mesmo sendo executado pela JVM, o Java utiliza técnicas como compilação Just-In-Time (JIT), que melhora a velocidade de execução dos programas.
- ❁ **Execução de múltiplas tarefas:** O Java permite executar várias tarefas ao mesmo tempo (multithreading), tornando os programas mais eficientes.
- ❁ **Robustez:** É uma linguagem fortemente tipada, o que ajuda a identificar erros ainda durante o desenvolvimento.
- ❁ **Ampla ecossistema:** Possui muitas bibliotecas, ferramentas e documentação, facilitando o aprendizado e o desenvolvimento de aplicações.

1.3 Estrutura de um Programa em Java

Todo programa em Java segue uma estrutura básica. Essa estrutura define como o código deve ser organizado para que o compilador consiga entender e executar corretamente as instruções programadas. Os programas devem ser organizados em uma estrutura hierárquica composta por **classes**, **métodos** e **blocos de código**.

- ❖ **Classe:** É um molde ou modelo que define a estrutura de um programa. Em Java, todo o código deve estar dentro de uma classe.
- ❖ **Métodos:** São blocos de código estruturados dentro de uma classe, projetados para realizar uma tarefa ou mais tarefas específicas.
- ❖ **Blocos de Código:** São estruturas compostas por instruções que compõem a lógica de um programa. Enquanto a instrução é uma única ação executável, o bloco de código é um agrupamento de instruções.
- ❖ **Instruções:** São comandos executáveis, organizados de forma individual ou agrupada com o objetivo de controlar o fluxo do programa.

Exemplo 1.1

```
1 // Classe
2 public class meuPrograma {
3
4     // Método
5     public static void main(String[] args) {
6
7         // Bloco de código
8         System.out.println("Hello World!"); //Instrução
9
10    }
11 }
```

Observação

As **chaves {}** são fundamentais, indicam o início e o fim de cada estrutura. Elas definem onde começa e termina uma classe, um método ou um bloco de código. Um uso incorreto das chaves pode causar erros no programa ou alterar completamente seu funcionamento.

Capítulo 2 Comentários

São anotações, textos explicativos escritos no código do algoritmo ou do programa que **não são executados pelo compilador, são ignorados**. Eles servem como notas para ajudar programadores a entender, organizar e documentar o código, facilitando futuras manutenções e o trabalho em equipe. Os comentários podem ser utilizados para "desativar" temporariamente um trecho do programa.

Observação

Não use comentários para descrever o que a linha de código já diz claramente. Isso apenas aumenta o esforço de leitura sem agregar valor.

2.1 Comentário de Linha Única

É usado para explicações simples e curtas. Neste caso, o trecho do código começa com `//` e todo o texto que segue até o final da linha é afetado.

Exemplo 2.1

```
1 // Variável velocidade é do tipo int
2
3 int velocidade = 80; // Define a velocidade máxima permitida
```

2.2 Comentário de Múltiplas Linhas

Diferentemente dos comentários de linha única, eles abrangem várias linhas consecutivas, geralmente delimitados por caracteres especiais, como `/* ... */`, começam com `/*` e terminam com `*/`.

Exemplo 2.2

```
1 /*
2     Este bloco de código calcula o imposto
3     segundo as regras do ano de 2024.
4 */
5 double imposto = 0.15;
```

2.3 Comentário de Documentação (Doc Comment - Javadoc)

São blocos de texto estruturados de modo que ferramentas possam extraí-los para gerar documentação externa (como HTML ou XML). Ao contrário de comentários comuns (linhas e múltiplas linhas), eles servem para explicar o propósito de elementos como: classes, métodos e parâmetros, facilitando a compreensão de outros desenvolvedores.

Em Java, o comentário de documentação, chamado de **Javadoc** é delimitado por caracteres especiais como `/** ... */`, começam com `/**` e terminam com `*/` e possui tags (palavras) específicas para documentar parâmetros:

- ❁ **@param:** Documenta os parâmetros de um método, indicando o nome de cada parâmetro e sua descrição.
- ❁ **@return:** Descreve o valor retornado pelo método, explicando o que será devolvido após a execução.
- ❁ **@throws ou @exception:** Indica as exceções que o método pode lançar durante sua execução, junto com a condição em que isso ocorre.
- ❁ **@version:** Especifica a versão da classe, interface ou método, útil para o controle da evolução do código.
- ❁ **@since:** Informa a partir de qual versão o elemento (classe, método, etc.) está disponível.
- ❁ **@see:** Cria referências a outros elementos relacionados (classes, métodos ou documentos), facilitando a navegação na documentação.

Exemplo 2.3

```
1 /**
2  * Define o nome de um usuário.
3  * @param nome o nome a ser atribuído ao usuário
4  */
5 public void setName(String nome) {
6     this.nome = nome;
7 }
```

Exemplo 2.4

```
1 /**
2  * Retorna o saldo atual da conta.
3  * @return o valor do saldo disponível
4  */
5 public double getSaldo() {
6     return saldo;
7 }
```

Exemplo 2.5

```
1 /**
2  * Realiza o saque de um valor.
3  * @param valor valor a ser sacado
4  * @throws IllegalArgumentException se o valor for negativo
5  */
6 public void sacar(double valor) {
7     if (valor < 0) {
8         throw new IllegalArgumentException("Valor inválido");
9     }
10    saldo -= valor;
11 }
```

Exemplo 2.6

```
1 /**
2  * Classe que representa um produto.
3  * @version 1.0
4  */
5 public class Produto {
6 }
```

Exemplo 2.7

```
1 /**
2  * Método disponível a partir da versão 2.0.
3  * @since 2.0
4  */
5 public void novoMetodo() {
6     // implementação
7 }
```

Exemplo 2.8

```
1 /**
2  * Calcula o dobro de um número.
3  * @see Math#abs(int)
4  */
5 public int dobro(int x) {
6     return x * 2;
7 }
```

Exemplo 2.9

```
1 /**
2  * Classe que representa uma calculadora simples.
3  * @version 1.0
4  * @since 2026
5  */
6 public class Calculadora {
7     /**
8      * Calcula a soma de dois números.
9      * @param a o primeiro número, b o segundo número
10     * @return a soma de a e b
11     * @throws IllegalArgumentException se algum valor for negativo
12     * @see Math#addExact(int, int)
13     */
14     public int somar(int a, int b) {
15         if (a < 0 || b < 0) {
16             throw new IllegalArgumentException("Valores não podem ser negativos");
17         }
18         return a + b;
19     }
20 }
```

2.4 Diferença entre comentários comuns e Javadoc

Os comentários comuns servem apenas para leitura no código, enquanto os comentários de documentação (Javadoc), além de documentar o código, podem gerar documentação oficial.

Capítulo 3 Variáveis

São espaços na memória do computador usados para armazenar informações. Uma variável é como uma "caixa" onde guardamos um valor para usar depois em um programa.

Cada variável possui um nome e um tipo, que indica o tipo de informação que ela pode armazenar, por exemplo, números ou textos. Em Java, as variáveis podem ser classificadas em dois tipos principais: (i) Tipos primitivos e (ii) Tipos por referência.

3.1 Tipos Primitivos

Os tipos primitivos são utilizados para armazenar valores diretamente na memória, como números, caracteres e valores lógicos. Java possui 8 tipos primitivos, organizados conforme a tabela abaixo:

Tabela 3.1. Tipos Primitivos em Java

Tipo	Descrição	Intervalo de valores	Exemplo
byte	Inteiro pequeno	-128 a 127	byte x = 10;
short	Inteiro curto	-32.768 a 32.767	short x = 100;
int	Inteiro padrão	-2.147.483.648 a 2.147.483.647	int x = 1000;
long	Inteiro grande	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807	long x = 10000L;
float	Decimal simples	$\pm 3.4 \times 10^{38}$ (aprox.)	float x = 10.5f;
double	Decimal preciso	$\pm 1.7 \times 10^{308}$ (aprox.)	double x = 10.5;
char	Caractere	0 a 65.535 (Unicode)	char c = 'A';
boolean	Lógico	true ou false	boolean b = true;

Exemplo 3.1

```
1 public class TiposPrimitivos {  
2  
3     public static void main(String[] args) {  
4         // Inteiros  
5         int idade = 18;  
6         int quantidade = 100;  
7         // Números reais (double)  
8         double preco = 19.99;  
9         double altura = 1.75;  
10        // Caractere  
11        char letra = 'A';  
12        char sexo = 'M';  
13        // Booleano  
14        boolean ativo = true;  
15        boolean aprovado = false;  
16        // Número real (float)  
17        float temperatura = 36.5f;  
18        float nota = 7.5f;  
    }
```

```
19 // Inteiro longo
20 long populacao = 8000000000L;
21 // Inteiro curto
22 short ano = 2024;
23 // Inteiro muito pequeno
24 byte nivel = 25;
25
26 // Impressão dos valores
27 System.out.println(idade);
28 System.out.println(quantidade);
29 System.out.println(preco);
30 System.out.println(altura);
31 System.out.println(letra);
32 System.out.println(sexo);
33 System.out.println(ativo);
34 System.out.println(aprovado);
35 System.out.println(temperatura);
36 System.out.println(nota);
37 System.out.println(populacao);
38 System.out.println(ano);
39 System.out.println(nivel);
40 }
41 }
```

3.2 Tipos por Referência (Não Primitivos)

Os tipos por referência, não primitivos, não armazenam diretamente o valor em si, mas sim uma referência (endereço) que aponta para a posição na memória onde o dado está armazenado. Esses tipos são utilizados para representar estruturas mais complexas, como **objetos**, **arrays** e **strings** (textos), permitindo manipular dados de forma mais flexível e dinâmica.

Tabela 3.2. Tipos por Referência em Java

Tipo	Descrição	Exemplo
String	Cadeia de caracteres	String nome = "João";
Array	Estrutura de vários elementos do mesmo tipo	int[] v = {1,2,3};
Classe (Objeto)	Representa objetos criados pelo programador	Pessoa p = new Pessoa();
Interface	Define comportamentos (contrato)	List l = new ArrayList();
Enum	Conjunto de constantes	Dia d = Dia.SEGUNDA;

Exemplo 3.2

```
1 public class ExemploReferencia {
2     // Classe (tipo por referência)
3     static class Pessoa {
4         String nome;
```

```
5     }
6
7     public static void main(String[] args) {
8         // String (objeto)
9         String nome = "Maria";
10        String cidade = "Belem";
11
12        // Arrays (estruturas por referência)
13        int[] numeros = {1, 2, 3, 4};
14        String[] nomes = {"Ana", "Joao", "Carlos"};
15
16        // Objeto da classe Pessoa
17        Pessoa p = new Pessoa();
18        p.nome = "Carlos";
19
20        // Impressões
21        System.out.println(nome);
22        System.out.println(cidade);
23        System.out.println(numeros[0]);
24        System.out.println(nomes[1]);
25        System.out.println(p.nome);
26    }
27 }
```

Escopo das Variáveis

O escopo define onde a variável pode ser usada no programa. Ou seja, o "limite" onde ela existe.

1. Variável de Classe (ou Global), é declarada fora dos métodos e pode ser usada em toda a classe.

Exemplo 3.3

```
1 public class Exemplo {
2     static int idade = 15; // variável de classe
3
4     public static void main(String[] args) {
5         System.out.println(idade);
6     }
7
8 }
```

2. Variável de nível de método, é declarada dentro de um método e só pode ser usada dentro dele.

Exemplo 3.4

```
1 public static void main(String[] args) {
2     int idade = 15; // variável local
3     System.out.println(idade);
4 }
```

3. Em nível do bloco de código, se uma variável é definida/declarada em algum bloco de código, ela existe até o final desse bloco de código. No exemplo abaixo, a variável só existe dentro do bloco `if`.

Exemplo 3.5

```
1 public static void main(String[] args) {  
2     if (true) {  
3         int numero = 10;  
4         System.out.println(numero); // funciona  
5     }  
6  
7 }
```

3.3 Conversão de Tipos (Casting)

É o processo de converter um valor de um tipo de dado para outro. Essa conversão é comum quando trabalhamos com diferentes tipos de variáveis ou precisamos adaptar um valor a um contexto específico. Por exemplo, podemos converter um número inteiro para um número real, ou um número real para inteiro. Existem dois tipos principais de casting em Java:

- ❖ **Casting Implícito (Widening)**
- ❖ **Casting Explícito (Narrowing)**

3.3.1 Casting Implícito

O casting implícito ocorre automaticamente quando convertemos um tipo de **menor capacidade** para um tipo de **maior capacidade**. Nesse caso, não há risco de perda de dados, pois o tipo de destino consegue armazenar o valor original.

Exemplo 3.6

```
1 public class CastingImplicito {  
2     public static void main(String[] args) {  
3         int numero = 10;  
4         // Casting implícito: int -> double  
5         double valor = numero;  
6  
7         System.out.println("Valor inteiro: " + numero);  
8         System.out.println("Valor convertido (double): " + valor);  
9     }  
10 }
```

3.4 Casting Explícito

O casting explícito ocorre quando convertemos um tipo de **maior capacidade** para um tipo de **menor capacidade**. Nesse caso, é necessário indicar a conversão manualmente, pois pode haver **perda de dados**.

Exemplo 3.7

```

1 public class CastingExplicito {
2
3     public static void main(String[] args) {
4
5         double valor = 9.78;
6
7         // Casting explícito: double - int
8         // A parte decimal é perdida na conversão.
9         int numero = (int) valor;
10
11         System.out.println("Valor original (double): " + valor);
12         System.out.println("Valor convertido (int): " + numero);
13     }
14 }

```

👉 Observação

O tipo de dado boolean é o único tipo primitivo que não suporta casting.

Segue abaixo uma tabela com todos os tipos de casting possíveis.

DE \ PARA	<i>byte</i>	<i>short</i>	<i>char</i>	<i>int</i>	<i>long</i>	<i>float</i>	<i>double</i>
<i>byte</i>	—	Implícito	Implícito	Implícito	Implícito	Implícito	Implícito
<i>short</i>	Explícito (perda)	—	Implícito	Implícito	Implícito	Implícito	Implícito
<i>char</i>	Explícito (perda)	Explícito (perda)	—	Implícito	Implícito	Implícito	Implícito
<i>int</i>	Explícito (perda)	Explícito (perda)	Explícito (perda)	—	Implícito	Implícito	Implícito
<i>long</i>	Explícito (perda)	Explícito (perda)	Explícito (perda)	Explícito (perda)	—	Implícito	Implícito
<i>float</i>	Explícito (perda)	Explícito (perda)	Explícito (perda)	Explícito (perda)	Explícito (perda)	—	Implícito
<i>double</i>	Explícito (perda)	Explícito (perda)	Explícito (perda)	Explícito (perda)	Explícito (perda)	Explícito (perda)	—

Implícito	Conversão implícita (automática) – não há perda de dados.
Explícito (perda)	Conversão explícita – pode haver perda de dados.
—	Mesmo tipo – não há conversão.

Implícito (widening): de tipos menores para maiores.
Ex.: `int i = 10; double d = i;`

Explícito (narrowing): de tipos maiores para menores.
Ex.: `double d = 9.78; int i = (int) d;`

Atenção: conversões explícitas podem causar perda de precisão.

Figura 3.1. Tabela completa de casting entre tipos primitivos em Java, destacando conversões implícitas e explícitas.

3.4.1 Conversão de Strings para Tipos Primitivos

É comum receber dados no formato de `String`, especialmente quando utilizamos entrada de dados (como `Scanner` ou interfaces gráficas). Para realizar cálculos ou operações lógicas, é necessário converter essas `Strings` em tipos primitivos. Essa conversão é feita por meio de métodos específicos.

Exemplo 3.8

```
1 public class ConversaoString {
2
3     public static void main(String[] args) {
4
5         // Strings de entrada
6         String textoInt = "10";
7         String textoDouble = "9.5";
8         String textoFloat = "7.5";
9         String textoBoolean = "true";
10        String textoChar = "Java";
11
12        // Conversões
13        int numero = Integer.parseInt(textoInt);
14        double valorDouble = Double.parseDouble(textoDouble);
15        float valorFloat = Float.parseFloat(textoFloat);
16        boolean ativo = Boolean.parseBoolean(textoBoolean);
17        char letra = textoChar.charAt(0);
18
19        // Saídas
20        System.out.println("Inteiro + 5: " + (numero + 5));           // 15
21        System.out.println("Double * 2: " + (valorDouble * 2));      // 19.0
22        System.out.println("Float + 1: " + (valorFloat + 1));        // 8.5
23        System.out.println("Booleano: " + ativo);                   // true
24        System.out.println("Primeira letra: " + letra);               // J
25    }
26 }
```

3.4.2 Conversão de Tipos Primitivos para String

É comum converter valores de tipos primitivos para `String`, especialmente quando precisamos exibir dados ou concatenar valores com textos. Essa conversão pode ser feita de diferentes formas.

Os tipos primitivos podem ser convertidos para `String` utilizando o método `valueOf()` ou `toString()`.

Exemplo 3.9

```
1 public class ConversaoParaString {
2
3     public static void main(String[] args) {
4
5         int numero = 10;
6         double valor = 9.5;
7         boolean ativo = true;
8         char letra = 'A';
9
10        // Conversões com valueOf
11        String s1 = String.valueOf(numero);
12        String s2 = String.valueOf(valor);
13
14        // Conversões com toString (wrapper)
```

```
15 String s3 = Integer.toString(numero);
16 String s4 = Double.toString(valor);
17
18 // Conversão automática (concatenação)
19 String s5 = ativo + "";
20 String s6 = letra + "";
21
22 // Impressão
23 System.out.println("valueOf: " + s1 + ", " + s2);
24 System.out.println("toString: " + s3 + ", " + s4);
25 System.out.println("Concatenação: " + s5 + ", " + s6);
26 }
27 }
```

3.5 Exercícios Resolvidos

Exercício Resolvidos 3.1

```
1 public class ExemploTipos {
2     public static void main(String[] args) {
3
4         int idade = 18;
5         double altura = 1.75;
6         char sexo = 'M';
7         boolean ativo = true;
8         float nota = 7.5f;
9         long populacao = 8000000L;
10        short ano = 2024;
11        byte nivel = 1;
12
13        System.out.println("Idade: " + idade);
14        System.out.println("Altura: " + altura);
15        System.out.println("Sexo: " + sexo);
16        System.out.println("Ativo: " + ativo);
17        System.out.println("Nota: " + nota);
18        System.out.println("Populacao: " + populacao);
19        System.out.println("Ano: " + ano);
20        System.out.println("Nivel: " + nivel);
21    }
22 }
```


Capítulo 4 Operadores

São símbolos ou palavras-chave especiais que indicam ao compilador ou ao interpretador que realize ações específicas, como cálculos matemáticos, comparações ou operações lógicas sobre dados, variáveis ou valores. Sua função principal é manipular dados e controlar o fluxo de um algoritmo. Veremos um conjunto de 5 grupos de operadores comumente utilizados na linguagem de programação Java.

4.1 Operadores Aritméticos

São utilizados para realizar operações aritméticas entre valores e variáveis.

Tabela 4.1. Operadores Aritméticos em Java

Operador	Nome	Descrição	Exemplo
+	Soma	Adiciona dois valores	10 + 5 = 15
-	Subtração	Subtrai um valor do outro	10 - 5 = 5
*	Multiplicação	Multiplica dois valores	10 * 5 = 50
/	Divisão	Divide um valor por outro	10 / 5 = 2
%	Módulo (resto)	Retorna o resto da divisão	10 % 3 = 1

Exemplo 4.1

```
1 public class OperadoresAritmeticos {
2     public static void main(String[] args) {
3         int numero1 = 20, numero2 = 4;
4         // Adição
5         int soma = numero1 + numero2;
6         // Subtração
7         int diferenca = numero1 - numero2;
8         // Multiplicação
9         int multiplicacao = numero1 * numero2;
10        // Divisão
11        int divisao = numero1 / numero2;
12        // Módulo (resto da divisão)
13        int resto = numero1 % numero2;
14
15        System.out.println("Soma: " + soma);
16        System.out.println("Diferença: " + diferenca);
17        System.out.println("Multiplicação: " + multiplicacao);
18        System.out.println("Divisão: " + divisao);
19        System.out.println("Resto: " + resto);
20    }
21 }
```

4.2 Operadores de Atribuição

São operadores que atribuem ou imputam um valor a uma variável e permitem definir um valor inicial ou atualizar o valor já existente, podendo também realizar operações durante a atribuição.

Tabela 4.2. Operadores Básicos de Atribuição em Java

Operador	Nome	Descrição	Exemplo
=	Atribuição simples	Define o valor à direita para a variável à esquerda.	x = 10
+=	Atribuição de adição	Soma o valor à direita ao valor atual da variável e atribui o resultado. Equivalente a x = x + 5.	x += 5
-=	Atribuição de subtração	Subtrai o valor à direita do valor atual da variável. Equivalente a x = x - 3.	x -= 3
*=	Atribuição de multiplicação	Multiplica o valor atual da variável pelo valor à direita. Equivalente a x = x * 2.	x *= 2
/=	Atribuição de divisão	Divide o valor atual da variável pelo valor à direita. Equivalente a x = x / 4.	x /= 4
%=	Atribuição de módulo (resto)	Calcula o resto da divisão entre o valor atual e o valor à direita, atribuindo o resultado. Equivalente a x = x % 3.	x %= 3

Exemplo 4.2

```

1 public class OperadoresAtribuicao {
2     public static void main(String[] args) {
3         int x = 10;
4         System.out.println("Valor inicial: " + x);
5         x += 5;
6         System.out.println("x += 5 -> " + x);
7         x -= 3;
8         System.out.println("x -= 3 -> " + x);
9         x *= 2;
10        System.out.println("x *= 2 -> " + x);
11        x /= 4;
12        System.out.println("x /= 4 -> " + x);
13        x %= 3;
14        System.out.println("x %= 3 -> " + x);
15    }
16 }

```

4.3 Operadores Relacionais ou de Comparação

Este grupo de operadores serve para analisar a relação entre valores ou variáveis. O resultado de uma operação de comparação é sempre um valor booleano: verdadeiro (true) ou falso (false).

Tabela 4.3. Operadores de Comparação em Java

Operador	Nome	Descrição	Exemplo
==	Igual	Verifica se dois valores são iguais	a == b
!=	Diferente	Verifica se os valores são diferentes	a != b
>	Maior que	Verifica se o primeiro valor é maior	a > b
<	Menor que	Verifica se o primeiro valor é menor	a < b
>=	Maior ou igual	Verifica se é maior ou igual	a >= b
<=	Menor ou igual	Verifica se é menor ou igual	a <= b

Exemplo 4.3

```
1 public class Comparacao {
2
3     public static void main(String[] args) {
4
5         int x = 10;
6
7         int y = 5;
8
9         System.out.println(x == y); // Resultado - false
10
11        System.out.println(x != y); // Resultado - true
12
13        System.out.println(x > y); // Resultado - true
14
15        System.out.println(x < y); // Resultado - false
16
17        System.out.println(x >= y); // Resultado - true
18
19        System.out.println(x <= y); // Resultado - false
20    }
21
22 }
```

4.4 Operadores Unários

São operadores que exigem apenas um único operando (variável ou valor) para realizar operações como incremento, decremento, inversão de sinal ou negação lógica. Eles agem diretamente sobre o operando para modificar seu valor ou estado, simplificando tarefas aritméticas e lógicas básicas.

Tabela 4.4. Operadores Unários em Java

Operador	Nome	Descrição	Exemplo
++	Incremento	Aumenta o valor em 1	x++ ou ++x
--	Decremento	Diminui o valor em 1	x-- ou --x
+	Positivo	Mantém o valor positivo	+x
-	Negação aritmética	Inverte o sinal do valor	-x
!	Negação lógica	Inverte o valor booleano	!true = false

Exemplo 4.4

```

1 public class ExemploUnarios {
2     public static void main(String[] args) {
3         int x = 5;
4         int y = 5;
5         int z = 5;
6
7         boolean status = true;
8
9         // Incremento
10        System.out.println("Pós-incremento (x++): " + (x++)); // usa e depois
incrementa
11
12        System.out.println("Valor de x após incremento: " + x);
13
14        System.out.println("Pré-incremento (++y): " + (++y)); // incrementa antes de
usar
15
16        // Decremento
17        System.out.println("Pós-decremento (z--): " + (z--)); // usa e depois
decrementa
18
19        System.out.println("Valor de z após decremento: " + z);
20
21        System.out.println("Pré-decremento (--z): " + (--z)); // decrementa antes de
usar
22
23        // Positivo (não altera)
24        int a = -10;
25        System.out.println("Positivo (+a): " + (+a));
26
27        // Negação aritmética
28        int b = 10;
29        System.out.println("Negação (-b): " + (-b));
30
31        // Negação lógica
32        System.out.println("Negação lógica (!status): " + (!status));
33    }
34 }

```

Atenção a condição de **Pré e Pós-Incremento**. São formas de utilizar o operador de incremento (++) e decremento (-). A principal diferença está na ordem em que a operação ocorre:

- ✿ ++x → incrementa primeiro e depois utiliza o valor.
- ✿ x++ → utiliza primeiro e depois incrementa.
- ✿ --x → decrementa primeiro e depois utiliza o valor.
- ✿ x-- → utiliza primeiro e depois decrementa.

Exemplo 4.5

```

1 public class ExemploIncremento {
2     public static void main(String[] args) {
3         int x = 5;
4         // pré-incremento
5         int a = ++x; // x = 5 + 1 = 6 -> a recebe 6
6         // pós-incremento
7         int b = x++; // b recebe 6 -> depois x = 6 + 1 = 7
8         System.out.println(x); // Resultado: 7
9         System.out.println(a); // Resultado: 6
10        System.out.println(b); // Resultado: 6
11    }
12 }

```

Passo a passo:

- ✿ ++x → primeiro, o valor de x é incrementado em 1. Como x valia 5, passa a valer 6. Em seguida, esse novo valor é atribuído à variável **a**. Portanto, **a = 6**.
- ✿ x++ → primeiro, o valor atual de x é utilizado. Como x valia 6, esse valor é atribuído à variável **b**. Depois disso, x é incrementado em 1, passando a valer 7. Portanto, **b = 6** e **x = 7**.

4.5 Operadores Lógicos

São símbolos utilizados para combinar ou inverter expressões booleanas (true ou false), permitindo explorar a tomada de decisão em estruturas condicionais (por exemplo: if, while). Eles avaliam as condições e retornam um único valor booleano com base nas regras de "E", "OU" e "NÃO".

Tabela 4.5. Operadores Lógicos em Java

Operador	Nome	Descrição	Exemplo
&&	E lógico (AND)	Retorna verdadeiro se ambas as condições forem verdadeiras	<code>x < 5 && x < 10</code>
	OU lógico (OR)	Retorna verdadeiro se pelo menos uma das condições for verdadeira	<code>x < 5 x < 4</code>
!	NÃO lógico (NOT)	Inverte o resultado lógico (true vira false e vice-versa)	<code>!(x < 5 && x < 10)</code>

Exemplo 4.6

```

1 public class ExemploOperadoresLogicos {
2     public static void main(String[] args) {
3         int x = 7;
4         // Operador AND (&&)
5         System.out.println("x > 5 && x < 10: " + (x > 5 && x < 10));
6         // Operador OR (||)
7         System.out.println("x < 5 || x < 10: " + (x < 5 || x < 10));
8         // Operador NOT (!)
9         System.out.println("!(x > 5 && x < 10): " + !(x > 5 && x < 10));
10    }
11 }

```

👉 Observação

As operações envolvendo os operadores lógicos, &&, || e !, seguem as regras da **tabela verdade**. A tabela verdade é a representação das possíveis combinações de valores lógicos (verdadeiro ou falso) e seus respectivos resultados.

4.5.1 Tabelas Verdade

Operador AND (&&) - Uma condição só é verdadeira se todas as condições forem verdadeiras.

Tabela 4.6. Tabela verdade do operador lógico AND (&&)

Condição A	Condição B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

Exemplo 4.7

```

1 public class OperadoresLogicos {
2     public static void main(String[] args) {
3         // =====
4         // Operador AND (&&)
5         // =====
6         boolean a = true;
7         boolean b = false;
8         System.out.println("AND (&&):");
9         System.out.println("true && true = " + (true && true)); // true
10        System.out.println("true && false = " + (true && false)); // false
11        System.out.println("false && true = " + (false && true)); // false
12        System.out.println("false && false = " + (false && false)); // false
13    }
14 }

```

Operador OR (| |) - Uma condição é verdadeira se pelo menos uma condição for verdadeira.

Tabela 4.7. Tabela verdade do operador lógico OR (| |)

Condição A	Condição B	A B
true	true	true
true	false	true
false	true	true
false	false	false

Exemplo 4.8

```

1 public class OperadoresLogicos {
2     public static void main(String[] args) {
3         // =====
4         // Operador OR (||)
5         // =====
6         System.out.println("\nOR (||):");
7         System.out.println("true || true = " + (true || true)); // true
8         System.out.println("true || false = " + (true || false)); // true
9         System.out.println("false || true = " + (false || true)); // true
10        System.out.println("false || false = " + (false || false)); // false
11    }
12 }
```

Operador NOT (!) - Inverte o valor lógico do resultado.

Tabela 4.8. Tabela verdade do operador lógico NOT (!)

Condição A	!A
true	false
false	true

Exemplo 4.9

```

1 public class OperadoresLogicos {
2     public static void main(String[] args) {
3         // =====
4         // Operador NOT (!)
5         // =====
6         System.out.println("\nNOT (!):");
7         System.out.println("!true = " + (!true)); // false
8         System.out.println("!false = " + (!false)); // true
9     }
10 }
```

4.6 Precedência de Operadores

A precedência de operadores define a ordem em que as operações são avaliadas em uma expressão. Operadores com maior precedência são executados primeiro. Quando necessário, podemos utilizar parênteses para alterar essa ordem.

Tabela 4.9. Tabela de precedência de operadores em Java (do maior para o menor nível)

Nível	Operadores	Descrição	Associatividade
1	() [] .	Parênteses, acesso a array, acesso a membros	Esq. → Dir.
2	++ - ! ~ + - (tipo)	Incremento/decremento, negação, cast	Dir. → Esq.
3	* / %	Multiplicação, divisão, resto	Esq. → Dir.
4	+ -	Soma e subtração	Esq. → Dir.
5	« » »>	Operadores de deslocamento (shift)	Esq. → Dir.
6	< <= > >= instanceof	Relacionais	Esq. → Dir.
7	== !=	Igualdade	Esq. → Dir.
8	&	AND lógico/bit a bit	Esq. → Dir.
9	^	XOR (ou exclusivo)	Esq. → Dir.
10		OR lógico/bit a bit	Esq. → Dir.
11	&&	AND lógico (curto-circuito)	Esq. → Dir.
12		OR lógico (curto-circuito)	Esq. → Dir.
13	?:	Operador ternário	Dir. → Esq.
14	= += -= *= /= %= &= ^= = «= »= »>=	Atribuição	Dir. → Esq.

Observação

- ❁ Operadores com maior precedência são avaliados primeiro.
- ❁ A associatividade define a direção da avaliação.
- ❁ Parênteses podem ser usados para alterar a ordem de execução.
- ❁ O uso de parênteses melhora a legibilidade do código.

4.7 Exercícios Resolvidos

Exercício Resolvidos 4.1

```

1 public class OperadoresLogicosTeste {
2     public static void main(String[] args) {
3
4         int idade = 20;
5         boolean temCarteira = true;
6         // AND: precisa atender as duas condições

```

```
7      if (idade >= 18 && temCarteira) {  
8          System.out.println("\nPode dirigir (AND)");  
9      }  
10     // OR: precisa atender pelo menos uma condição  
11     if (idade >= 18 || temCarteira) {  
12         System.out.println("Atende pelo menos uma condição (OR)");  
13     }  
14     // NOT: inverte a condição  
15     if (!temCarteira) {  
16         System.out.println("Não possui carteira (NOT)");  
17     }  
18 }  
19 }
```


Capítulo 5 Estruturas Condicionais

São blocos de código que permitem ao programa tomar decisões e alterar seu fluxo de execução com base em condições lógicas (verdadeiro ou falso). Os principais tipos de estruturas condicionais são: if, if-else, if-else if-else e switch.

5.1 If (Simples)

A estrutura condicional `if` é a forma mais básica de tomada de decisão. Ela permite que um bloco de código seja executado apenas quando uma condição lógica for avaliada como verdadeira (`true`). Caso a condição seja falsa (`false`), o bloco é ignorado e o fluxo do programa segue normalmente para as próximas instruções.

A condição utilizada no `if` deve sempre resultar em um valor booleano, ou seja, verdadeiro ou falso.

Exemplo 5.1

```
1 public class ExemploIfSimples {
2     public static void main(String[] args) {
3         int idade = 18;
4         if (idade >= 18) {
5             System.out.println("Maior de idade");
6         }
7         System.out.println("Fim do programa");
8     }
9 }
```

5.2 If ... else (Composta)

A estrutura condicional `if ... else` permite ao programa escolher entre dois caminhos possíveis, com base no resultado de uma condição lógica. Se a condição for verdadeira (`true`), o bloco associado ao `if` será executado. Caso contrário, se a condição for falsa (`false`), o bloco associado ao `else` será executado.

Exemplo 5.2

```
1 public class ExemploIfElse {
2     public static void main(String[] args) {
3         int idade = 16;
4         if (idade >= 18) {
5             System.out.println("Maior de idade");
6         } else {
7             System.out.println("Menor de idade");
8         }
9         System.out.println("Verificação concluída");
10    }
11 }
```

5.3 If ... else if ... else (Encadeado)

A estrutura condicional `if...else if...else` é utilizada quando há necessidade de testar múltiplas condições de forma sequencial. O programa avalia cada condição na ordem em que elas aparecem. Ao encontrar a primeira condição verdadeira, executa o bloco correspondente e ignora os demais.

Se nenhuma das condições for verdadeira, o bloco `else` será executado.

Exemplo 5.3

```
1 public class ExemploIfElseIf {
2     public static void main(String[] args) {
3         int nota = 5;
4         if (nota >= 7) {
5             System.out.println("Aprovado");
6         } else if (nota >= 5) {
7             System.out.println("Recuperacao");
8         } else {
9             System.out.println("Reprovado");
10        }
11        System.out.println("Resultado final exibido");
12    }
13 }
```

5.4 Switch-case

A estrutura `switch-case` é utilizada para selecionar uma entre várias alternativas, com base no valor de uma variável. Ela é especialmente indicada quando há múltiplas comparações de igualdade envolvendo o mesmo valor, tornando o código mais organizado e legível em comparação com vários `if-else`.

Cada opção é representada por um `case`, e o comando `break` é utilizado para encerrar a execução daquele bloco, evitando que os próximos casos também sejam executados. O bloco `default` é opcional e será executado caso nenhum dos casos anteriores seja atendido.

Exemplo 5.4

```
1 public class ExemploSwitchCase {
2     public static void main(String[] args) {
3
4         int dia = 3;
5
6         switch (dia) {
7             case 1:
8                 System.out.println("Domingo");
9                 break;
10            case 2:
11                System.out.println("Segunda-feira");
12                break;
13            case 3:
14                System.out.println("Terca-feira");
```

```
15         break;
16     default:
17         System.out.println("Outro dia");
18     }
19
20     System.out.println("Fim da verificacao");
21 }
22 }
```

No exemplo acima, a variável dia possui valor 3, o programa executa o bloco correspondente ao case 3, exibindo "Terça-feira". Após executar a instrução print, o comando break encerra o fluxo do programa continua normalmente. Caso nenhum valor corresponda aos casos definidos, o bloco default será executado.

5.4.1 Tipos de Dados Permitidos

A estrutura switch-case permite selecionar uma alternativa com base no valor de uma variável. No entanto, nem todos os tipos de dados podem ser utilizados nessa estrutura. São aceitos os seguintes tipos de dados:

- ✿ int, byte e short
- ✿ char
- ✿ String (a partir do Java 7)
- ✿ enum

Exemplo 5.5

```
1 public class ExemploSwitchInt {
2     public static void main(String[] args) {
3
4         int opcao = 2;
5
6         switch (opcao) {
7             case 1:
8                 System.out.println("Opcao 1 selecionada");
9                 break;
10            case 2:
11                System.out.println("Opcao 2 selecionada");
12                break;
13            case 3:
14                System.out.println("Opcao 3 selecionada");
15                break;
16            default:
17                System.out.println("Opcao invalida");
18        }
19    }
20 }
```

Exemplo 5.6

```
1 public class ExemploSwitchChar {
2     public static void main(String[] args) {
3
4         char letra = 'A';
5
6         switch (letra) {
7             case 'A':
8                 System.out.println("Letra A selecionada");
9                 break;
10            case 'B':
11                System.out.println("Letra B selecionada");
12                break;
13            case 'C':
14                System.out.println("Letra C selecionada");
15                break;
16            default:
17                System.out.println("Letra nao reconhecida");
18        }
19    }
20 }
21
22 }
```

Exemplo 5.7

```
1 public class ExemploSwitchString {
2     public static void main(String[] args) {
3
4         String dia = "segunda";
5
6         switch (dia) {
7             case "segunda":
8                 System.out.println("Inicio da semana");
9                 break;
10            case "terca":
11                System.out.println("Segundo dia da semana");
12                break;
13            case "sexta":
14                System.out.println("Fim da semana");
15                break;
16            default:
17                System.out.println("Dia invalido");
18        }
19    }
20 }
21
22 }
```

5.4.2 Tipos Não Permitidos

Os seguintes tipos de dados, não são permitidos e não podem ser utilizados no switch:

- ✿ boolean
- ✿ float
- ✿ double
- ✿ long

Exemplo 5.8

```
1 public class ExemploSwitchInvalido {  
2     public static void main(String[] args) {  
3  
4         boolean ativo = true;  
5  
6         // O código abaixo gera erro de compilação  
7         switch (ativo) { // ERRO!  
8             case true:  
9                 System.out.println("Ativo");  
10                break;  
11             case false:  
12                System.out.println("Inativo");  
13                break;  
14            }  
15        }  
16    }
```

Exemplo 5.9

```
1 public class ExemploIfElseBoolean {  
2     public static void main(String[] args) {  
3  
4         boolean ativo = true;  
5  
6         if (ativo) {  
7             System.out.println("Ativo");  
8         } else {  
9             System.out.println("Inativo");  
10        }  
11    }  
12 }
```

Em resumo, os tipos de dados suportados pela linguagem que podem ser utilizados na estrutura switch-case apresentados na Tabela abaixo.

Tabela 5.1. Tipos de dados permitidos no switch-case

Tipo de Dado	Permitido no switch?
int / byte / short	Sim
char	Sim
String	Sim
enum	Sim
boolean	Não
float / double	Não
long	Não

5.5 Exercícios Resolvidos

Exercício Resolvidos 5.1

```
1 public class ExemploIf {  
2  
3     public static void main(String[] args) {  
4  
5         int x = 20;  
6         int y = 18;  
7  
8         if (x > y) {  
9             System.out.println("x é maior que y");  
10        }  
11    }  
12 }
```

Exercício Resolvidos 5.2

```
1 public class ParOuImpar {  
2  
3     public static void main(String[] args) {  
4  
5         int myNum = 5;  
6  
7         if (myNum % 2 == 0) {  
8             System.out.println(myNum + " é par");  
9         } else {  
10            System.out.println(myNum + " é ímpar");  
11        }  
12    }  
13 }
```

Exercício Resolvidos 5.3

```
1 import javax.swing.JOptionPane;
2 public class Porta {
3
4     public static void main(String[] args) {
5
6         int doorCode = Integer.parseInt(
7             JOptionPane.showInputDialog("Digite o código da porta:");
8         );
9
10        if (doorCode == 1337) {
11            JOptionPane.showMessageDialog(null, "Código correto. Porta Aberta.");
12        } else {
13            JOptionPane.showMessageDialog(null, "Código incorreto. Porta Fechada.");
14        }
15    }
16 }
```

Exercício Resolvidos 5.4

```
1 import javax.swing.JOptionPane;
2
3 public class DiaSemana {
4     public static void main(String[] args) {
5
6         int dia = Integer.parseInt(
7             JOptionPane.showInputDialog("Digite um número de 1 a 7:");
8         );
9
10        if (dia == 1) {
11            JOptionPane.showMessageDialog(null, "Domingo");
12        } else if (dia == 2) {
13            JOptionPane.showMessageDialog(null, "Segunda-feira");
14        } else if (dia == 3) {
15            JOptionPane.showMessageDialog(null, "Terça-feira");
16        } else if (dia == 4) {
17            JOptionPane.showMessageDialog(null, "Quarta-feira");
18        } else if (dia == 5) {
19            JOptionPane.showMessageDialog(null, "Quinta-feira");
20        } else if (dia == 6) {
21            JOptionPane.showMessageDialog(null, "Sexta-feira");
22        } else if (dia == 7) {
23            JOptionPane.showMessageDialog(null, "Sábado");
24        } else {
25            JOptionPane.showMessageDialog(null, "Valor inválido");
26        }
27    }
28 }
```

Exercício Resolvidos 5.5

```
1 import javax.swing.JOptionPane;
2
3 public class SistemaUsuarios {
4     public static void main(String[] args) {
5
6         String entrada = JOptionPane.showInputDialog(
7             "Digite o tipo de usuario:\n1 - Aluno\n2 - Professor\n3 - Coordenador"
8         );
9
10        int opcao = Integer.parseInt(entrada);
11
12        switch (opcao) {
13            case 1:
14                JOptionPane.showMessageDialog(null, "Acesso ao sistema de estudos");
15                break;
16            case 2:
17                JOptionPane.showMessageDialog(null, "Acesso ao sistema de ensino");
18                break;
19            case 3:
20                JOptionPane.showMessageDialog(null, "Acesso completo ao sistema");
21                break;
22            default:
23                JOptionPane.showMessageDialog(null, "Usuario invalido");
24        }
25    }
26 }
```

Capítulo 6 Estruturas de Repetição

As estruturas de repetição permitem executar um bloco de código várias vezes, de forma automática, enquanto uma determinada condição for satisfeita. Essas estruturas são utilizadas para automatizar tarefas repetitivas, tornando os programas mais organizados, eficientes e reduzindo a quantidade de código. A linguagem Java possui três principais estruturas de repetição:

- ✿ for
- ✿ while
- ✿ do-while

Cada estrutura possui características específicas e pode ser utilizada de acordo com a necessidade do programa.

6.1 For

A estrutura de repetição for é utilizada quando sabemos previamente quantas vezes um bloco de código deverá ser executado. Ela organiza, em uma única linha, três etapas importantes do controle da repetição: inicialização, condição e incremento.

Exemplo 6.1

```
1 for (inicializacao; condicao; incremento) {  
2     // bloco de codigo a ser executado  
3 }
```

- ✿ **Inicialização:** é executada apenas uma vez, no início do laço. Nessa etapa, normalmente criamos e inicializamos a variável de controle que será utilizada durante as repetições.
- ✿ **Condição:** é verificada antes de cada repetição. Enquanto a condição for verdadeira, o bloco de código continuará sendo executado. Quando a condição se tornar falsa, o laço será encerrado.
- ✿ **Incremento:** é executado ao final de cada repetição. Geralmente, essa etapa altera o valor da variável de controle (inicialização) por meio de operações de incremento ou decremento.

Exemplo 6.2

```
1 public class ExemploFor {  
2     public static void main(String[] args) {  
3         for (int i = 1; i <= 3; i++) {  
4             System.out.println("Valor: " + i);  
5         }  
6         //Saida:  
7         //Valor: 1  
8         //Valor: 2  
9         //Valor: 3  
10    }  
11 }
```

6.2 While

A estrutura de repetição `while` é utilizada quando não sabemos previamente quantas vezes um bloco de código deverá ser executado. Ela executa repetidamente um conjunto de instruções enquanto uma determinada condição for verdadeira.

Em outras palavras, a estrutura continua sendo executada até que a condição se torne falsa. A estrutura do `while` é composta pelos seguintes elementos:

Exemplo 6.3

```
1 while (condicao) {  
2     // bloco de codigo a ser executado  
3 }
```

A **condição** é verificada antes de cada repetição do laço. Enquanto a condição for verdadeira, o bloco de código continuará sendo executado. Quando a condição se tornar falsa, o laço será encerrado.

Observação

Durante a execução do laço `while`, é comum usar uma instrução para alterar o valor da variável utilizada na condição. Essa alteração é importante para que, em determinado momento, a condição se torne falsa e a estrutura seja encerrada. Caso a condição nunca seja modificada, o programa poderá entrar em um **laço infinito**, executando repetidamente o mesmo bloco de código.

Exemplo 6.4

```
1 public class ExemploAlterandoCondicaoWhile {  
2     public static void main(String[] args) {  
3         int contador = 1;  
4         while (contador <= 5) {  
5             System.out.println("Contador: " + contador);  
6             // altera a variavel utilizada na condicao  
7             contador++;  
8         }  
9         //Saida:  
10        //Contador: 1  
11        //Contador: 2  
12        //Contador: 3  
13        //Contador: 4  
14        //Contador: 5  
15    }  
16 }
```

Observação

A estrutura `while` em Java aceita apenas uma expressão booleana, ou seja, uma condição que retorne `true` ou `false`. Diferente de outras linguagens, Java não permite utilizar diretamente valores numéricos, caracteres ou strings como condição do `while`. Além disso, é muito comum utilizar condições compostas por meio de operadores lógicos, o que permite verificar duas ou mais condições ao mesmo tempo.

Exemplo 6.5

```
1 public class ExemploWhileComCondicaoComposta {
2     public static void main(String[] args) {
3         int numero = 1;
4         int limite = 5;
5         while (numero <= limite && numero > 0) {
6             System.out.println("Numero: " + numero);
7             numero++;
8         }
9         /*
10        Saida:
11        Numero: 1
12        Numero: 2
13        Numero: 3
14        Numero: 4
15        Numero: 5
16        */
17    }
18 }
```

Exemplo 6.6

```
1 public class ExemploWhileOR {
2     public static void main(String[] args) {
3         int idade = 16;
4         boolean autorizado = true;
5
6         while (idade >= 18 || autorizado) {
7             System.out.println("Acesso permitido");
8             break;
9         }
10        // Saida:
11        //Acesso permitido
12    }
13 }
```

Exemplo 6.7

```
1 public class ExemploWhileNOT {
2     public static void main(String[] args) {
3         boolean ativo = false;
4
5         while (!ativo) {
6             System.out.println("Sistema inativo");
7             ativo = true;
8         }
9         // Saida:
10        //Sistema inativo
11    }
12 }
```

6.3 do-while

A estrutura de repetição do-while é semelhante ao while. No entanto, sua principal característica é garantir que o bloco de código seja executado pelo menos uma vez, pois a condição é verificada somente após a execução do bloco.

Diferentemente das estruturas for e while, nas quais a condição é testada antes da execução do laço, no do-while o teste ocorre ao final de cada repetição. A sintaxe básica da estrutura do-while é apresentada a seguir:

Exemplo 6.8

```
1 do {  
2     // bloco de código a ser executado  
3 } while (condicao);
```

O bloco de código é executado inicialmente e, em seguida, a condição é verificada. Se a condição for verdadeira, o laço continuará executando. Caso contrário, o laço será encerrado.

Exemplo 6.9

```
1 public class ExemploDoWhile {  
2     public static void main(String[] args) {  
3         int i = 1;  
4         do {  
5             System.out.println("Valor: " + i);  
6             i++;  
7         } while (i <= 3);  
8         //Saída:  
9         //Valor: 1  
10        //Valor: 2  
11        //Valor: 3  
12    }  
13 }
```

Mesmo que a condição seja falsa logo no início, o bloco de código será executado uma vez antes da verificação da condição.

Exemplo 6.10

```
1 public class ExemploDoWhileUmaVez {  
2     public static void main(String[] args) {  
3         int numero = 10;  
4         do {  
5             System.out.println("Executando o bloco");  
6         } while (numero < 5);  
7         //Saída:  
8         //Executando o bloco  
9     }  
10 }
```

6.4 Break

O comando `break` é utilizado para interromper imediatamente a execução de uma estrutura de repetição ou de seleção. Quando executado, o programa encerra a repetição (loop) ou o bloco atual e continua a execução a próxima instrução após a estrutura interrompida. O `break` pode ser utilizado nas estruturas:

- ✿ `for`
- ✿ `while`
- ✿ `do-while`
- ✿ `switch`

Exemplo 6.11

```
1 public class ExemploBreak {
2     public static void main(String[] args) {
3         for (int i = 1; i <= 10; i++) {
4             if (i == 4) {
5                 break;
6             }
7             System.out.println(i);
8         }
9         //Saída:
10        //1
11        //2
12        //3
13    }
14 }
```

O comando `break` também pode ser utilizado em estruturas `while`.

Exemplo 6.12

```
1 public class ExemploBreakWhile {
2     public static void main(String[] args) {
3         int contador = 1;
4         while (contador <= 10) {
5             if (contador == 5) {
6                 break;
7             }
8             System.out.println(contador);
9             contador++;
10        }
11        //Saída:
12        //1
13        //2
14        //3
15        //4
16    }
17 }
```

Na estrutura `switch`, o `break` é utilizado para impedir que os próximos casos sejam executados.

Exemplo 6.13

```
1 public class ExemploBreakSwitch {
2     public static void main(String[] args) {
3
4         int opcao = 2;
5
6         switch (opcao) {
7             case 1:
8                 System.out.println("Opcao 1");
9                 break;
10
11             case 2:
12                 System.out.println("Opcao 2");
13                 break;
14
15             case 3:
16                 System.out.println("Opcao 3");
17                 break;
18
19             default:
20                 System.out.println("Opcao invalida");
21         }
22         /*
23         Saida:
24         Opcao 2
25         */
26
27     }
28 }
```

** Observação**

O comando `break` interrompe apenas a estrutura em que ele foi executado. Em estruturas aninhadas, apenas o laço interno será encerrado.

Além disso, o uso excessivo do `break` pode dificultar a leitura e manutenção do código. Por isso, ele deve ser utilizado apenas quando realmente necessário.

6.5 Continue

Diferentemente do `break`, o comando `continue` não encerra a estrutura de repetição. Ele apenas pula a execução da instrução definida e continua normalmente para a próxima iteração. O `continue` pode ser utilizado nas estruturas:

- ✿ `for`
- ✿ `while`
- ✿ `do-while`

Exemplo 6.14

```
1 public class ExemploContinueFor {
2     public static void main(String[] args) {
3         for (int i = 1; i <= 5; i++) {
4             if (i == 3) {
5                 continue;
6             }
7             System.out.println(i);
8         }
9         //Saída:
10        //1
11        //2
12        //4
13        //5
14    }
15 }
```

O comando continue também pode ser utilizado na estrutura while.

Exemplo 6.15

```
1 public class ExemploContinueWhile {
2     public static void main(String[] args) {
3         int contador = 0;
4         while (contador < 10) {
5             contador++;
6             if (contador == 5) {
7                 continue;
8             }
9             System.out.println(contador);
10        }
11
12        /*
13        Saída:
14        1
15        2
16        3
17        4
18        6
19        7
20        8
21        9
22        10
23        */
24
25    }
26 }
```

Exemplo do comando continue na estrutura do-while.

Exemplo 6.16

```
1 public class ExemploContinueDoWhile {
2     public static void main(String[] args) {
3         int contador = 0;
4         do {
5             contador++;
6             if (contador == 3) {
7                 continue;
8             }
9             System.out.println(contador);
10        } while (contador < 6);
11
12        /*
13        Saída:
14        1
15        2
16        4
17        5
18        6
19        */
20    }
21 }
```

** Observação**

Ao utilizar o comando `continue` em estruturas `while` e `do-while`, é importante garantir que a variável de controle do loop seja alterada corretamente. Caso contrário, o programa poderá entrar em um laço infinito.

A Tabela abaixo apresenta as principais diferença entre `break` e `continue`.

Tabela 6.1. Diferença entre os comandos `break` e `continue`

Comando	Função
<code>break</code>	Encerra completamente o laço
<code>continue</code>	Interrompe apenas a iteração determinada

Capítulo 7 Classes e Objetos

A Programação Orientada a Objetos (POO) é um paradigma de programação baseado na criação e manipulação de objetos. Em Java tudo é organizado em classes e objetos.

Uma **classe** é um modelo (estrutura) que define as características e o comportamento de um objeto. Ela funciona como um “molde” para criar objetos.

Um **objeto** é uma instância (exemplo) de uma classe. Cada objeto possui seus próprios valores e pode executar ações definidas pela classe. De forma simples:

- ✿ Classe → Modelo
- ✿ Objeto → Instância (exemplo) criada a partir do modelo

Por exemplo:

- ✿ Classe: Carro
- ✿ Objetos: carro1, carro2, carro3

Todos os objetos da classe Carro possuem características semelhantes, como marca, modelo e cor, mas cada objeto pode ter valores diferentes.

7.1 Estrutura básica de uma classe

Uma classe em Java possui uma estrutura padrão utilizada para organizar os elementos. A estrutura básica de uma classe compreende.

Exemplo 7.1

```
1 "modificador_de_Acesso" "palavra_chave" "Nome_da_Classe"(){  
2     // corpo da classe  
3 }
```

- ✿ **Modificador de Acesso:** Define quem pode acessar a classe ou seus membros.
 - ✿ **public (Público):** É o modificador mais comum. Define que a classe ou membro pode ser acessado por qualquer outra classe, independentemente do pacote onde esteja.
 - ✿ **default (Padrão/Pacote):** Ocorre quando nenhum modificador é declarado. O acesso é permitido apenas para classes do mesmo pacote.
- ✿ **Palavra-chave "class":** Indica ao compilador Java que uma nova classe está sendo declarada.
- ✿ **Nome da Classe:** Representa o nome da classe criada. Por convenção, os nomes das classes utilizam o padrão *PascalCase*, iniciando cada palavra com letra maiúscula, como por exemplo: Carro, ContaBancaria e Aluno.
- ✿ **Corpo da Classe:** É a região delimitada pelas chaves { }. Dentro dela são declarados os atributos, métodos e demais elementos da classe.

Além disso, uma classe pode conter:

- ❖ Atributos: representam características do objeto.
- ❖ Métodos: representam ações ou comportamentos do objeto.

O exemplo abaixo apresenta uma classe com atributos e métodos.

Exemplo 7.2

```
1 public class Cachorro {  
2     // Atributos (O que o cachorro TEM)  
3     String nome;  
4     String raca;  
5     int idade;  
6     // Métodos (O que o cachorro Faz)  
7     void latir() {  
8         System.out.println(nome + " disse: Au Au!");  
9     }  
10 }
```

7.2 Criando Classes

Para criar uma classe em Java devemos definir: (i) modificador de acesso; (ii) adicionar a palavra-chave "class"; (iii) adicionar um nome à classe. Veja o exemplo abaixo.

Exemplo 7.3

```
1 //(i) modificador de acesso - public;  
2 //(ii) adicionar a palavra-chave - class;  
3 //(iii) adicionar um nome à classe - Carro;  
4 public class Carro {  
5  
6 }
```

7.3 Criando Objetos

Após definir uma classe, é possível criar objetos a partir dela. Um objeto é uma instância de uma classe, ou seja, uma estrutura concreta criada em memória, com seus próprios atributos e comportamentos.

Em Java, a criação de objetos é realizada utilizando a palavra-chave `new`. Essa palavra reserva espaço na memória para o objeto e chama o construtor da classe. A sintaxe básica para criação de um objeto é apresentada abaixo.

Exemplo 7.4

```
1 NomeDaClasse nomeObjeto = new NomeDaClasse();
```

- ❖ `NomeDaClasse`: representa a classe utilizada como modelo;
- ❖ `nomeObjeto`: representa o nome da variável de referência do objeto;
- ❖ `new`: cria o objeto em memória;
- ❖ `NomeDaClasse()`: chama o construtor da classe.

Exemplo 7.5

```
1 public class Carro {
2     public static void main(String[] args) {
3         // Criando objeto
4         Carro carro1 = new Carro();
5     }
6 }
```

O objeto criado pode acessar os atributos e métodos definidos na classe utilizando o operador ponto (.). Abaixo, temos um exemplo de criação de objetos e de acesso a métodos e atributos.

Exemplo 7.6

```
1 public class Carro {
2     //Atributos
3     String marca;
4     int ano;
5
6     //Metodo
7     void exibirInformacoes() {
8         System.out.println("Marca: " + marca);
9         System.out.println("Ano: " + ano);
10    }
11
12    public static void main(String[] args) {
13
14        // Criando objeto
15        Carro carro1 = new Carro();
16
17        // Alterando atributos
18        carro1.marca = "Toyota";
19        carro1.ano = 2023;
20
21        // Chamando metodo
22        carro1.exibirInformacoes();
23    }
24 }
25 //Saída esperada:
26
27 //Marca: Toyota
28 //Modelo: Corolla
29 //Ano: 2022
30
31 //Marca: Honda
32 //Modelo: Civic
33 //Ano: 2021
```

7.4 Modificadores de Acesso - Atributos e Métodos

Os modificadores de acesso em Java são utilizados para definir o nível de visibilidade de classes, atributos e métodos. Eles controlam quais partes do programa podem acessar determinados elementos. Os principais modificadores de acesso para atributos e métodos são:

- ❖ `public`
- ❖ `private`
- ❖ `protected`
- ❖ `default` (sem modificador)

Os modificadores são importantes para organizar o código e proteger os dados internos das classes, seguindo o princípio do encapsulamento.

7.4.1 Modificador `public`

O modificador `public` permite que atributos ou métodos sejam acessados por qualquer outra classe do programa, independentemente do pacote.

Exemplo 7.7

```
1 public class Pessoa {  
2  
3     public String nome;  
4  
5     public void apresentar() {  
6         System.out.println("Nome: " + nome);  
7     }  
8 }
```

No exemplo acima, o atributo `nome` e o método `apresentar()` também podem ser acessados livremente.

7.4.2 Modificador `private`

O modificador `private` restringe o acesso apenas à própria classe onde o elemento foi declarado. Esse modificador é muito utilizado para proteger atributos e impedir alterações diretas externas.

Exemplo 7.8

```
1 public class ContaBancaria {  
2  
3     private double saldo;  
4  
5     public void mostrarSaldo() {  
6         System.out.println("Saldo: " + saldo);  
7     }  
8 }
```

- ❖ O atributo `saldo` só pode ser acessado dentro da classe `ContaBancaria`;
- ❖ Outras classes não conseguem acessar diretamente esse atributo.

7.4.3 Modificador `protected`

Esse modificador é bastante utilizado em herança. Permite acesso:

- ❖ Nas classes do mesmo pacote;
- ❖ Nas subclasses, mesmo em pacotes diferentes.

Exemplo 7.9

```
1 public class Animal {
2     protected String nome;
3     protected void emitirSom() {
4         System.out.println("Som do animal");
5     }
6 }
```

Exemplo 7.10

```
1 public class Cachorro extends Animal {
2     void apresentar() {
3         // Acessando atributo protected
4         nome = "Rex";
5         System.out.println("Nome: " + nome);
6
7         // Chamando metodo protected
8         emitirSom();
9     }
10
11     public static void main(String[] args) {
12         Cachorro c1 = new Cachorro();
13         c1.apresentar();
14     }
15 }
16 //Saída Esperada:
17 //Nome: Rex
18 //O animal emitiu um som.
```

No exemplo acima, o atributo `nome` e o método `emitirSom()` podem ser acessados por subclasses da classe `Animal`.

7.4.4 Modificador `default`

Quando nenhum modificador é declarado, o elemento possui acesso padrão (`default`). Nesse caso, o acesso é permitido apenas para classes do mesmo pacote.

Exemplo 7.11

```

1 package loja;
2 class Produto {
3
4     String descricao = "Notebook";
5
6     void exibirDescricao() {
7         System.out.println("Produto: " + descricao);
8     }
9 }

```

Outra classe do mesmo pacote consegue acessar normalmente a classe Produto.

Exemplo 7.12

```

1 package loja;
2 public class Principal {
3
4     public static void main(String[] args) {
5
6         Produto p1 = new Produto();
7
8         p1.exibirDescricao();
9     }
10 }
11 //Saída Esperada
12 //Produto: Notebook

```

No exemplo acima, a classe Produto, o atributo descricao e o método exibirDescricao() só podem ser acessados por classes do mesmo pacote loja.

Em resumo, os modificadores permitem que atributos e métodos:

Tabela 7.1. Resumo dos Modificadores de Acesso

Modificador	Mesma Classe	Mesmo Pacote	Outros Pacotes
public	Sim	Sim	Sim
protected	Sim	Sim	Subclasses
default	Sim	Sim	Não
private	Sim	Não	Não

7.5 Modificadores de Acesso em Classes

Os modificadores de acesso também podem ser aplicados diretamente às classes. Nesse caso, eles definem quais classes do programa poderão acessar a classe criada. Em Java, classes podem utilizar apenas os modificadores:

❁ public

✿ default (sem modificador)

Os modificadores `private` e `protected` não podem ser aplicados em classes principais (*top-level classes*).

7.5.1 Classe `public`

Uma classe declarada como `public` pode ser acessada por qualquer outra classe do programa, inclusive em diferentes pacotes.

Exemplo 7.13

```
1 public class Pessoa {  
2  
3     String nome;  
4  
5     void apresentar() {  
6         System.out.println("Nome: " + nome);  
7     }  
8 }
```

7.5.2 Classe `default`

Quando nenhum modificador é declarado, a classe possui acesso padrão (`default`). Nesse caso, ela só poderá ser acessada por outras classes do mesmo pacote.

Exemplo 7.14

```
1 package loja;  
2 class Produto {  
3     String descricao = "Notebook";  
4  
5     void exibirDescricao() {  
6         System.out.println("Produto: " + descricao);  
7     }  
8 }
```

Como a classe `Principal` pertence ao mesmo pacote (`loja`), ela consegue acessar normalmente a classe `Produto`.

Exemplo 7.15

```
1 package loja;  
2 public class Principal {  
3     public static void main(String[] args) {  
4  
5         Produto p1 = new Produto();  
6         p1.exibirDescricao();  
7     }  
8 }
```

7.6 Exercícios Resolvidos

Exercício Resolvidos 7.1

Crie uma classe chamada Pessoa

✿ Declare dois atributos:

✿ nome do tipo String;

✿ idade do tipo int.

✿ Crie um método chamado apresentar() que exiba na tela o nome e a idade da pessoa.

✿ No método main(), crie um objeto da classe Pessoa utilizando a palavra-chave new.

✿ Atribua o nome "Maria" e a idade 20 ao objeto criado.

✿ Chame o método apresentar() para exibir os dados na tela.

```
1 public class Pessoa {
2
3     String nome;
4     int idade;
5     void apresentar() {
6         System.out.println("Nome: " + nome);
7         System.out.println("Idade: " + idade);
8     }
9
10    public static void main(String[] args) {
11        Pessoa p1 = new Pessoa();
12
13        p1.nome = "Maria";
14        p1.idade = 20;
15
16        p1.apresentar();
17    }
18 }
19
20 //Saída esperada:
21 //Nome: Maria
22 //Idade: 20
```

Capítulo 8 Métodos

É um bloco de código estruturado dentro de uma classe, projetado para executar uma tarefa específica e ser reutilizado. Ele define o comportamento de um objeto, podendo receber parâmetros, processar informações e retornar um resultado, o que facilita a organização, a manutenção e a leitura do código.

Sua estrutura simplificada é formada por:

Exemplo 8.1

```
1 "qualificador" "tipo-de-retorno" "nome-metodo" (Parametros){  
2     //código do corpo  
3 }
```

Qualificador: Define quem pode acessar o método (sua visibilidade).

- ❖ **public:** pode ser usado por qualquer classe.
- ❖ **private:** pode ser usado apenas dentro da própria classe.
- ❖ **protected:** pode ser usado pela própria classe, subclasses e classes do mesmo pacote.

Tipo de Retorno: Indica o tipo de dado que o método devolve após concluir sua execução.

- ❖ Quando o método não retorna nenhum valor, utiliza-se a palavra **void**.
- ❖ O valor retornado pode ser de um tipo primitivo: **byte**, **short**, **int**, **long**, **float**, **double**, **char** e **boolean**; um array ou até mesmo um objeto.

Nome do Método: É o nome que identifica o método.

- ❖ Deve começar com letra minúscula.
- ❖ Se tiver mais de uma palavra, as próximas começam com letra maiúscula (ex: **imprimeTexto**).
- ❖ Não pode conter espaços.
- ❖ Deve ser um nome claro, que indique o que o método faz.

Lista de Parâmetros: São variáveis passadas para o método executar sua tarefa. Também são denominados (argumentos).

- ❖ São opcionais (o método pode ou não ter parâmetros).
- ❖ Pode haver mais de um parâmetro (variável).
- ❖ Funciona como variáveis de entrada para o método.
- ❖ Esses valores podem ser usados no processamento para gerar resultados.

Observação

Para chamar um método, deve-se escrever o nome do método seguido de parênteses. Quando o método não é **static**, ele deve ser chamado a partir de um objeto.

De acordo com o **tipo de retorno**, os métodos podem ser classificados em: **sem retorno** ou **com retorno**. Além disso, ambos podem ser classificados em **com parâmetros** ou **sem parâmetros**.

Exemplo de método estático (sem objeto)

Exemplo 8.2

```
1 public class Calculadora {  
2  
3     public static void mensagem() {  
4         System.out.println("Calculadora pronta!");  
5     }  
6  
7     public static void main(String[] args) {  
8         Calculadora.mensagem(); // sem criar objeto  
9     }  
10 }
```

Exemplo de método com objeto para métodos não estáticos.

Exemplo 8.3

```
1 public class Pessoa {  
2  
3     public void saudacao() {  
4         System.out.println("Olá!");  
5     }  
6  
7     public static void main(String[] args) {  
8         Pessoa p = new Pessoa(); // criação do objeto  
9         p.saudacao(); // chamada do método  
10    }  
11 }
```

8.1 Métodos sem Retorno

São métodos que executam uma ação, mas não retornam nenhum valor ao final. Utilizam a palavra-chave `void`. A estrutura geral é:

Exemplo 8.4

```
1 "qualificador" "void" "nomeDoMetodo"(Parametros){  
2     // código do corpo  
3 }
```

Os métodos sem retorno de valores podem ser definidos **com parâmetros** ou **sem parâmetros**.

8.1.1 Métodos sem retorno e com parâmetros

São métodos que recebem variáveis (parâmetros), executam o processamento, mas não retornam valores.

Exemplo 8.5

```
1 public class MetodosSemRetorno {
2     public static void imprimirTexto(String texto) {
3         System.out.println(texto);
4     }
5     public static void somar(int a, int b) {
6         System.out.println(a + b);
7     }
8     public static void main(String[] args) {
9         imprimirTexto("Olá");
10        somar(10, 20);
11    }
12 }
```

8.1.2 Métodos sem retorno e sem parâmetros

São métodos que não recebem variáveis de entrada para processamento.

Exemplo 8.6

```
1 public class MetodosSemRetorno {
2     public static void imprimir() {
3         System.out.println("Aprendendo Java");
4     }
5     public static void main(String[] args) {
6         imprimir();
7     }
8 }
```

8.2 Métodos com Retorno de Valores

Os métodos com retorno de valores devolvem um valor ao final de sua execução. Isso é feito por meio da instrução `return`. Estes seguem a seguinte estrutura:

Exemplo 8.7

```
1 "qualificador" "tipoDoRetorno" "nomeDoMetodo"([listaDeParametros]){
2     // código do corpo
3     return valorDeRetorno;
4 }
```

Essa estrutura é semelhante à dos métodos sem retorno, com a diferença de que não há a palavra `void` e o valor é devolvido por meio da instrução **return** ao final do método.

** Observação**

Após o `return`, indicar o valor a ser retornado, geralmente uma variável ou uma expressão.

Os métodos com retorno de valores podem ser definidos **com parâmetros** ou **sem parâmetros**.

8.2.1 Métodos com retorno e com parâmetros

São métodos que recebem variáveis de entrada (parâmetros), processam esses valores e retornam um resultado.

Exemplo 8.8

```
1 public class Operacoes {
2
3     public static int somar(int a, int b) {
4         return a + b;
5     }
6     public static int quadrado(int numero) {
7         return numero * numero;
8     }
9     public static String verificarPar(int numero) {
10        if (numero % 2 == 0) {
11            return "Par";
12        } else {
13            return "Ímpar";
14        }
15    }
16    public static void main(String[] args) {
17        System.out.println(somar(5, 3));
18        System.out.println(quadrado(4));
19        System.out.println(verificarPar(7));
20    }
21 }
```

8.2.2 Métodos com retorno e sem parâmetros

São métodos que não recebem dados de entrada, mas ainda assim retornam um valor.

Exemplo 8.9

```
1 public class ExemploRetorno {
2     public static int obterNumero() {
3         return 10;
4     }
5     public static String saudacao() {
6         return "Olá, seja bem-vindo!";
7     }
8     public static void main(String[] args) {
9         System.out.println(obterNumero());
10        System.out.println(saudacao());
11    }
12 }
```

Exercício Resolvidos 8.1

```
1 public class MetodosComRetorno {
2     public static void main(String[] args) {
3         System.out.println(somar(10, 20));
4         System.out.println(quadrado(5));
5         System.out.println(maior(3, 8));
6     }
7     public static int somar(int a, int b) {
8         return a + b;
9     }
10    public static int quadrado(int numero) {
11        return numero * numero;
12    }
13    public static int maior(int a, int b) {
14        if (a > b) {
15            return a;
16        } else {
17            return b;
18        }
19    }
20 }
```

Exercício Resolvidos 8.2

```
1 public class AreaComSobrecarga {
2     public static void main(String args[]) {
3         System.out.println("Área de um quadrado: " + area(3));
4         System.out.println("Área de um retângulo: " + area(3, 2));
5         System.out.println("Área de um cubo: " + area(3, 2, 5));
6     }
7     public static double area(int x) {
8         return (x * x);
9     }
10    public static double area(int x, int y) {
11        return (x * y);
12    }
13    public static double area(int x, int y, int z) {
14        return (x * y * z);
15    }
16 }
```

8.3 Exercícios Práticos**Exercício 8.1****Questão – Método sem retorno com parâmetros**

Crie um método chamado `dobro(int numero)` que receba um número inteiro e mostre na tela o dobro desse valor.

Exercício 8.2**Questão – Método sem retorno sem parâmetros**

Crie um método chamado `mensagem()` que exiba na tela a frase: *"Bem-vindo ao Java!"*. Chame esse método no `main`.

Exercício 8.3**Questão - Método com retorno com parâmetros**

Crie um método chamado `multiplicar(int a, int b)` que receba dois números e retorne o resultado da multiplicação.

Exercício 8.4**Questão - Método com retorno sem parâmetros**

Crie um método chamado `obterNumero()` que retorne o número 100. Mostre esse valor no `main`.

Exercício 8.5**Questão Sobrecarga de métodos (soma)**

Implemente dois métodos com o nome `somar`: um que receba dois números inteiros e outro que receba três números inteiros. Em seguida, chame os dois métodos no `main` e exiba os resultados.

Exercício 8.6**Questão - Sobrecarga de métodos (tipos diferentes)**

Implemente dois métodos com o nome `multiplicar`: um que receba dois valores do tipo `int` e outro que receba dois valores do tipo `double`. No método `main`, chame ambos e apresente os resultados.

Exercício 8.7**Cálculo de Frete com Taxa Administrativa**

A transportadora **LogExpress** necessita de um novo módulo para seu sistema de faturamento. Sua tarefa é desenvolver um programa que calcule o custo final de transporte de mercadorias.

? Requisitos de Negócio:

- 1. Entrada de Dados:** O programa deve solicitar ao usuário o *peso da carga* (P) em quilogramas e o *valor do frete por quilo* (V_k).
- 2. Regra de Cálculo:** O valor total do frete (T) é composto pelo custo variável somado a uma taxa administrativa fixa de **R\$ 25,00**. A fórmula a ser aplicada é:

$$T = (P \times V_k) + 25.00$$

- 3. Validação:** O sistema não deve aceitar valores negativos ou nulos para o peso ou para o valor unitário. Caso ocorra, uma mensagem de erro deve ser exibida.
- 4. Saída de Dados:** O resultado final deve ser exibido com o prefixo da moeda brasileira (R\$) e formatado obrigatoriamente com duas casas decimais.

Exercício 8.8

Questão - Sobrecarga de Métodos Crie uma classe chamada `CaixaEletronico` que gerencie o saldo de uma conta. A classe deve conter:

- ✿ Um atributo `saldo`.
- ✿ Um método `sacar(double valor)`: Este é o saque para **clientes comuns**. Ele deve descontar o valor do saque mais uma taxa fixa de **0,5%**.
- ✿ Uma sobrecarga do método `sacar(double valor, boolean ehEspecial)`: Se o parâmetro `ehEspecial` for `true`, a taxa aplicada deve ser de apenas **0,1%**.
- ✿ Um método para exibir o saldo atual.

Capítulo 9 Construtores

Ao criar um objeto em Java, normalmente é necessário inicializar seus atributos com valores iniciais. Para essa finalidade, a linguagem disponibiliza um mecanismo especial denominado **construtor**.

Um construtor é um método especial executado automaticamente no momento da criação de um objeto. Sua principal função é inicializar os atributos da classe e preparar o objeto para uso. Um construtor possui as seguintes características:

- ❁ Possui o mesmo nome da classe;
- ❁ Não possui tipo de retorno, nem mesmo `void`;
- ❁ É executado automaticamente quando um objeto é criado;
- ❁ Pode receber parâmetros;
- ❁ Pode ser sobrecarregado.

A sintaxe básica é apresentada a seguir:

Exemplo 9.1

```
1 class Pessoa {  
2  
3     public Pessoa() {  
4  
5     }  
6  
7 }
```

Observe que o método possui exatamente o mesmo nome da classe. O construtor padrão é aquele que não recebe parâmetros.

Exemplo 9.2

```
1 class Pessoa {  
2  
3     String nome;  
4  
5     //Metodo com o mesmo nome da classe  
6     public Pessoa() {  
7         nome = "Maria";  
8     }  
9  
10    public void apresentar() {  
11        System.out.println(nome);  
12    }  
13 }
```

Programa principal:

Exemplo 9.3

```
1 public class Main {  
2  
3     public static void main(String[] args) {  
4  
5         Pessoa p = new Pessoa();  
6  
7         p.apresentar();  
8     }  
9 }  
10 //Saída:  
11 //Maria
```

9.1 A Palavra-chave this

A palavra-chave `this` é utilizada para referenciar os atributos do objeto atual. Considere o exemplo:

Exemplo 9.4

```
1 class Pessoa {  
2  
3     String nome;  
4     int idade;  
5  
6     public Pessoa(String nome, int idade) {  
7         this.nome = nome;  
8         this.idade = idade;  
9     }  
10  
11     public void apresentar() {  
12         System.out.println("Nome: " + nome);  
13         System.out.println("Idade: " + idade);  
14     }  
15  
16 }
```

Nesse caso:

- ✿ `this.nome` representa o atributo da classe;
- ✿ `nome` representa o parâmetro recebido pelo construtor.

Sem o uso de `this`, haveria ambiguidade entre o atributo e o parâmetro.

9.2 Construtor com Parâmetros

Em muitos casos, deseja-se fornecer valores no momento da criação do objeto.

Exemplo 9.5

```
1 class Pessoa {
2
3     String nome;
4     int idade;
5
6     public Pessoa(String nome, int idade) {
7         this.nome = nome;
8         this.idade = idade;
9     }
10
11     public void apresentar() {
12         System.out.println("Nome: " + nome);
13         System.out.println("Idade: " + idade);
14     }
15
16 }
```

9.3 Sobrecarga de Construtores

Uma mesma classe pode possuir vários construtores, desde que possuam listas de parâmetros diferentes. Essa técnica é denominada **sobrecarga de construtores**.

Exemplo 9.6

```
1 class Pessoa {
2
3     String nome;
4     int idade;
5
6     public Pessoa() {
7         nome = "Nao informado";
8         idade = 0;
9     }
10
11     public Pessoa(String nome) {
12         this.nome = nome;
13         idade = 0;
14     }
15
16     public Pessoa(String nome, int idade) {
17         this.nome = nome;
18         this.idade = idade;
19     }
20
21 }
```

Utilização:

Exemplo 9.7

```
1 public class Main {
2
3     public static void main(String[] args) {
4
5         Pessoa p1 = new Pessoa();
6
7         Pessoa p2 = new Pessoa("Ana");
8
9         Pessoa p3 = new Pessoa("Carlos", 30);
10    }
11
12 }
```

9.4 Chamando um Construtor a partir de Outro

Java permite que um construtor invoque outro construtor da mesma classe utilizando `this()`.

Exemplo 9.8

```
1 class Pessoa {
2
3     String nome;
4     int idade;
5
6     public Pessoa() {
7         this("Nao informado", 0);
8     }
9
10    public Pessoa(String nome, int idade) {
11        this.nome = nome;
12        this.idade = idade;
13    }
14
15 }
```

9.5 Construtores e Herança

Quando uma classe herda de outra, o construtor da superclasse pode ser chamado utilizando a palavra-chave `super()`. Classe base:

Exemplo 9.9

```
1 class Pessoa {
2
3     protected String nome;
4
5     public Pessoa(String nome) {
6         this.nome = nome;
7     }
8 }
```

```
7     }  
8  
9 }
```

Classe derivada:

Exemplo 9.10

```
1 class Professor extends Pessoa {  
2  
3     private String disciplina;  
4  
5     public Professor(String nome, String disciplina) {  
6  
7         super(nome);  
8  
9         this.disciplina = disciplina;  
10    }  
11  
12 }
```

Nesse caso, o construtor da classe Pessoa é executado antes do construtor da classe Professor.

Capítulo 10 Sobrecarga

A sobrecarga de métodos ocorre quando uma mesma classe possui dois ou mais métodos com o mesmo nome, mas com listas de parâmetros diferentes. Essa diferença pode estar na quantidade, no tipo ou na ordem dos parâmetros. Essa técnica permite usar um único nome de método para representar operações semelhantes, tornando o código mais organizado e intuitivo, permitindo a reutilização de nomes de métodos.

Observação

Apenas alterar o tipo de retorno do método não caracteriza sobrecarga. É necessário modificar os parâmetros.

Exemplo 10.1

```
1 public class ExemploSobrecarga {
2     public static void main(String[] args) {
3         System.out.println(somar(2, 3));
4         System.out.println(somar(2, 3, 4));
5     }
6
7     public static int somar(int a, int b) {
8         return a + b;
9     }
10
11    public static int somar(int a, int b, int c) {
12        return a + b + c;
13    }
14
15 }
```

Exemplo 10.2

```
1 public class ExemploSobrecarga2 {
2     public static int somar(int a, int b) {
3         return a + b;
4     }
5     public static double somar(double a, double b) {
6         return a + b;
7     }
8     public static void main(String[] args) {
9         System.out.println(somar(5, 2));
10        System.out.println(somar(5.5, 2.5));
11    }
12 }
```

10.1 Exercícios Resolvidos

Exercício Resolvidos 10.1

```
1 public class MetodosSemRetorno {
2     public static void main(String args[]) {
3         imprimir();
4         imprimirTexto("Ola");
5         quadrado(10);
6         somar(10, 20);
7         maior(10, 20, 30);
8         sexo('F');
9     }
10    public static void imprimir() {
11        System.out.println("Aprendendo a Linguagem Java");
12    }
13    public static void imprimirTexto(String texto) {
14        System.out.println(texto);
15    }
16    public static void somar(int a, int b) {
17        System.out.println(a + b);
18    }
19    public static void quadrado(int a) {
20        System.out.println(Math.pow(a, 2));
21    }
22    public static void maior(int a, int b, int c) {
23        System.out.println(Math.max(c, Math.max(a, b)));
24    }
25    public static void sexo(char c) {
26        if (c == 'F') {
27            System.out.println("Feminino");
28        } else if (c == 'M') {
29            System.out.println("Masculino");
30        } else {
31            System.out.println("Sexo desconhecido");
32        }
33    }
34 }
```

Capítulo 11 Herança

A herança é um dos principais conceitos da Programação Orientada a Objetos (POO). Em Java, ela permite que uma classe aproveite atributos e métodos de outra classe. A classe que fornece os atributos e métodos é chamada de **superclasse**, **classe pai** ou **classe base**. A classe que recebe essas características é chamada de **subclasse**, **classe filha** ou **classe derivada**.

Em Java, a herança é feita com a palavra-chave `extends`. A herança permite reutilizar atributos e métodos de uma classe existente ao criar uma nova classe.

11.1 Sintaxe da Herança

A estrutura básica da herança em Java é apresentada a seguir.

Exemplo 11.1

```
1 class ClassePai {
2     // atributos e metodos da classe pai
3 }
4
5 class ClasseFilha extends ClassePai {
6     // atributos e metodos da classe filha
7 }
```

Nesse caso, a `ClasseFilha` herda os atributos e métodos acessíveis da `ClassePai`.

O exemplo a seguir apresenta uma classe `Pessoa` e uma classe `Aluno`. Como todo aluno também é uma pessoa, a classe `Aluno` pode herdar características da classe `Pessoa`.

Exemplo 11.2

```
1 class Pessoa {
2
3     protected String nome;
4     protected int idade;
5
6     public void apresentar() {
7         System.out.println("Nome: " + nome);
8         System.out.println("Idade: " + idade);
9     }
10 }
11
12 class Aluno extends Pessoa {
13
14     private String matricula;
15
16     public void definirDados(String nome, int idade, String matricula) {
17         this.nome = nome;
18         this.idade = idade;
19         this.matricula = matricula;
20     }
21 }
```

```
21
22     public void exibirMatricula() {
23         System.out.println("Matricula: " + matricula);
24     }
25 }
26
27 public class ExemploHeranca {
28     public static void main(String[] args) {
29
30         Aluno aluno = new Aluno();
31
32         aluno.definirDados("Ana", 18, "2024001");
33
34         aluno.apresentar();
35         aluno.exibirMatricula();
36     }
37 }
38
39 Saída:
40
41     Nome: Ana
42     Idade: 18
43     Matricula: 2024001
```

No exemplo, a classe `Aluno` herda os atributos `nome` e `idade`, além do método `apresentar()` da classe `Pessoa`. A classe `Aluno` também possui um atributo próprio, chamado `matricula`.

O modificador `protected` permite que atributos e métodos sejam acessados pela própria classe, por classes do mesmo pacote e por subclasses. No exemplo anterior, os atributos `nome` e `idade` foram declarados como `protected` para que a classe `Aluno` pudesse acessá-los diretamente.

Caso esses atributos fossem declarados como `private`, a classe filha não poderia acessá-los diretamente.

11.2 Vantagens da Herança

A herança permite reutilizar código, organizar classes de forma hierárquica e evitar repetição de atributos e métodos comuns. Por exemplo, se várias classes representam pessoas, como `Aluno`, `Professor` e `Funcionario`, todas podem herdar características gerais da classe `Pessoa`.

Exemplo 11.3

```
1 class Pessoa {
2
3     protected String nome;
4     protected int idade;
5
6     public void apresentar() {
7         System.out.println("Nome: " + nome);
8         System.out.println("Idade: " + idade);
```

```
9     }
10 }
11
12 class Aluno extends Pessoa {
13
14     private String matricula;
15
16     public void definirDados(String nome, int idade, String matricula) {
17
18         this.nome = nome;
19         this.idade = idade;
20         this.matricula = matricula;
21     }
22
23     public void exibirMatricula() {
24         System.out.println("Matricula: " + matricula);
25     }
26 }
27
28 class Professor extends Pessoa {
29
30     private String disciplina;
31
32     public void definirDados(String nome, int idade, String disciplina) {
33
34         this.nome = nome;
35         this.idade = idade;
36         this.disciplina = disciplina;
37     }
38
39     public void exibirDisciplina() {
40         System.out.println("Disciplina: " + disciplina);
41     }
42 }
43
44 class Funcionario extends Pessoa {
45
46     private String cargo;
47     private double salario;
48
49     public void definirDados(String nome, int idade, String cargo, double salario) {
50
51         this.nome = nome;
52         this.idade = idade;
53         this.cargo = cargo;
54         this.salario = salario;
55     }
56
57     public void exibirDadosProfissionais() {
58         System.out.println("Cargo: " + cargo);
```

```
59     System.out.println("Salario: R$ " + salario);
60 }
61 }
62
63 public class ExemploHeranca {
64
65     public static void main(String[] args) {
66
67         Aluno aluno = new Aluno();
68         aluno.definirDados("Joao", 19, "2025001");
69
70         Professor professor = new Professor();
71         professor.definirDados("Carlos", 42, "Programacao Java");
72
73         Funcionario funcionario = new Funcionario();
74         funcionario.definirDados("Maria", 35, "Secretaria", 3500.00);
75
76         System.out.println("=== Aluno ===");
77         aluno.apresentar();
78         aluno.exibirMatricula();
79
80         System.out.println();
81
82         System.out.println("=== Professor ===");
83         professor.apresentar();
84         professor.exibirDisciplina();
85
86         System.out.println();
87
88         System.out.println("=== Funcionario ===");
89         funcionario.apresentar();
90         funcionario.exibirDadosProfissionais();
91     }
92 }
```

Nesse caso, a classe Professor reutiliza os atributos e métodos da classe Pessoa, mas também adiciona informações específicas, como a disciplina ministrada.

11.3 Herança e a palavra-chave final

Quando uma classe é declarada com a palavra-chave `final`, ela não pode ser herdada por outra classe. Isso significa que nenhuma classe poderá usar `extends` com uma classe marcada como `final`.

Exemplo 11.4

```
1 final class Pessoa {
2     // codigo da classe
3 }
4
```

```
5 // Erro: uma classe final nao pode ser herdada
6 class Aluno extends Pessoa {
7     // codigo da classe
8 }
```

Observação

Em Java, uma classe pode herdar diretamente apenas uma classe. Isso significa que Java não permite herança múltipla de classes.

Exemplo 11.5

```
1 // Forma permitida
2 class Aluno extends Pessoa {
3 }
4
5 // Forma nao permitida em Java
6 class Aluno extends Pessoa, Usuario {
7 }
```


Capítulo 12 Polimorfismo

O polimorfismo é um dos pilares da Programação Orientada a Objetos. O termo polimorfismo significa literalmente *muitas formas* . Em Java, ele permite que um mesmo método apresente comportamentos diferentes dependendo do objeto que o executa.

O polimorfismo está diretamente relacionado à herança. Quando uma classe filha sobrescreve um método da classe pai, cada objeto pode executar sua própria versão desse método. Essa característica torna os programas mais flexíveis, reutilizáveis e fáceis de manter.

Considere uma classe `Animal` que possui um método chamado `emitirSom()`.

Exemplo 12.1

```
1 class Animal {
2
3     public void emitirSom() {
4         System.out.println("O animal emitu um som.");
5     }
6 }
```

As classes filhas podem sobrescrever esse método para apresentar comportamentos específicos.

Exemplo 12.2

```
1 class Cachorro extends Animal {
2
3     @Override
4     public void emitirSom() {
5         System.out.println("Au Au!");
6     }
7 }
8
9 class Gato extends Animal {
10
11     @Override
12     public void emitirSom() {
13         System.out.println("Miau!");
14     }
15 }
```

Nesse caso, embora todas as classes possuam um método chamado `emitirSom()`, cada uma executa uma ação diferente. Vamos analisar o exemplo completo.

Exemplo 12.3

```
1 class Animal {
2
3     public void emitirSom() {
4         System.out.println("Som generico");
5     }
6 }
```

```
6 }
7
8 class Cachorro extends Animal {
9
10     @Override
11     public void emitirSom() {
12         System.out.println("Au Au!");
13     }
14 }
15
16 class Gato extends Animal {
17
18     @Override
19     public void emitirSom() {
20         System.out.println("Miau!");
21     }
22 }
23
24 public class ExemploPolimorfismo {
25
26     public static void main(String[] args) {
27
28         Animal animal1 = new Cachorro();
29         Animal animal2 = new Gato();
30
31         animal1.emitirSom();
32         animal2.emitirSom();
33     }
34 }
35
36 //Saída
37 //Au Au!
38 //Miau!
```

Observe que as variáveis foram declaradas como do tipo `Animal`, mas armazenam objetos das classes `Cachorro` e `Gato`. Quando o método `emitirSom()` é chamado, o Java executa automaticamente a versão correspondente ao tipo real do objeto.

Esse comportamento é chamado de **polimorfismo em tempo de execução** (*Runtime Polymorphism*).

12.1 Vantagens do Polimorfismo

O uso do polimorfismo oferece diversas vantagens:

- ✿ Reutilização de código;
- ✿ Maior flexibilidade na construção de sistemas;
- ✿ Facilidade de manutenção;
- ✿ Redução de estruturas condicionais complexas;
- ✿ Melhor organização das classes.

Imagine um sistema acadêmico que possui diferentes tipos de pessoas.

Exemplo 12.4

```
1 class Pessoa {
2
3     public void apresentar() {
4         System.out.println("Sou uma pessoa.");
5     }
6 }
7
8 class Aluno extends Pessoa {
9
10    @Override
11    public void apresentar() {
12        System.out.println("Sou um aluno.");
13    }
14 }
15
16 class Professor extends Pessoa {
17
18    @Override
19    public void apresentar() {
20        System.out.println("Sou um professor.");
21    }
22 }
23
24 class Funcionario extends Pessoa {
25
26    @Override
27    public void apresentar() {
28        System.out.println("Sou um funcionario.");
29    }
30 }
```

Utilizando polimorfismo:

Exemplo 12.5

```
1 public class SistemaAcademico {
2
3     public static void main(String[] args) {
4
5         Pessoa[] pessoas = {
6             new Aluno(),
7             new Professor(),
8             new Funcionario()
9         };
10
11         for (Pessoa pessoa : pessoas) {
12             pessoa.apresentar();
13         }
14     }
15 }
```

```
14     }  
15 }  
16 //Saida  
17 //Sou um aluno.  
18 //Sou um professor.  
19 //Sou um funcionario.
```

Nesse exemplo, o mesmo método `apresentar()` assume diferentes comportamentos dependendo do objeto armazenado no array.

12.2 A Anotação @Override

A anotação `@Override` indica que um método está sobrescrevendo um método herdado da superclasse. Embora seu uso seja opcional, ela é altamente recomendada, pois ajuda o compilador a identificar possíveis erros.

Exemplo 12.6

```
1 @Override  
2 public void apresentar() {  
3     System.out.println("Sou um aluno.");  
4 }
```

Observação

O polimorfismo permite que objetos de diferentes classes sejam tratados através de uma referência comum, normalmente uma superclasse. Quando um método é chamado, o Java determina automaticamente qual implementação deve ser executada com base no tipo real do objeto.

Capítulo 13 Classes Abstratas

Tem como principal objetivo servir como modelo para outras classes por meio da herança, ou seja, representa um conceito genérico que será especializado por outras classes. Ela pode conter:

- ❁ Atributos;
- ❁ Métodos concretos (com implementação);
- ❁ Métodos abstratos (sem implementação);
- ❁ Construtores.

A sintaxe básica envolve a declaração da palavra-chave `abstract`

Exemplo 13.1

```
1 abstract class NomeClasse {  
2  
3 }
```

Por exemplo, considere um sistema escolar. Existem alunos, professores e funcionários. Todos possuem nome e idade, porém cada um possui características específicas (especializações). Nesse caso, pode-se criar uma classe abstrata chamada `Pessoa` (generalização).

Exemplo 13.2

```
1 abstract class Pessoa {  
2     protected String nome;  
3     protected int idade;  
4 }
```

A partir dessa classe, outras podem ser especializadas:

Exemplo 13.3

```
1 class Aluno extends Pessoa {  
2  
3 }  
4  
5 class Professor extends Pessoa {  
6  
7 }  
8  
9 class Funcionario extends Pessoa {  
10  
11 }
```

As classes abstratas oferecem diversas vantagens:

- ❁ Evitam a criação de objetos genéricos que não fazem sentido no sistema;
- ❁ Promovem reutilização de código;

- ❖ Facilitam a manutenção do software;
- ❖ Permitem definir comportamentos obrigatórios para as subclasses;
- ❖ Organizam melhor a hierarquia de herança.

13.1 Métodos Abstratos

Uma das principais características das classes abstratas é a possibilidade de definir **métodos abstratos**. Um método abstrato possui apenas sua assinatura, sem implementação. Quando uma classe possui um método abstrato, ela obrigatoriamente deve ser declarada como abstrata, para isso devemos usar o nome **abstract**.

Exemplo 13.4

```
1 abstract class Pessoa {  
2  
3     //Método sem implementação, não tem corpo  
4     public abstract void exibirDados();  
5  
6 }
```

Toda classe que herdar de Pessoa deverá implementar esse método. O exemplo a seguir apresenta uma hierarquia simples utilizando classes abstratas.

Exemplo 13.5

```
1 abstract class Pessoa {  
2  
3     protected String nome;  
4     protected int idade;  
5  
6     public Pessoa(String nome, int idade) {  
7         this.nome = nome;  
8         this.idade = idade;  
9     }  
10  
11     public void apresentar() {  
12         System.out.println("Nome: " + nome);  
13         System.out.println("Idade: " + idade);  
14     }  
15  
16     //Deve ser obrigatoriamente implementada nas funções que herdam a classe pessoa (  
17     //especializadas)  
18     public abstract void exibirFuncao();  
19 }
```

Classe Professor, estende a classe Pessoa:

Exemplo 13.6

```
1 class Professor extends Pessoa {
2
3     private String disciplina;
4
5     public Professor(String nome, int idade, String disciplina) {
6         super(nome, idade);
7         this.disciplina = disciplina;
8     }
9
10    @Override
11    public void exibirFuncao() {
12        System.out.println("Professor de " + disciplina);
13    }
14 }
```

Classe Aluno:

Exemplo 13.7

```
1 class Aluno extends Pessoa {
2
3     private String matricula;
4
5     public Aluno(String nome, int idade, String matricula) {
6         super(nome, idade);
7         this.matricula = matricula;
8     }
9
10    @Override
11    public void exibirFuncao() {
12        System.out.println("Matricula: " + matricula);
13    }
14 }
```

Programa principal:

Exemplo 13.8

```
1 public class Main {
2
3     public static void main(String[] args) {
4
5         Professor prof = new Professor("Carlos", 45, "Programacao");
6
7         Aluno aluno = new Aluno("Ana", 20, "202501");
8
9         prof.apresentar();
10        prof.exibirFuncao();
11    }
```

```
12     System.out.println();
13
14     aluno.apresentar();
15     aluno.exibirFuncao();
16 }
17 }
```

13.2 Classe Abstrata versus Classe Comum

A Tabela 13.1 apresenta as principais diferenças entre uma classe abstrata e uma classe comum.

Tabela 13.1. Classe comum versus classe abstrata

Classe Comum	Classe Abstrata
Pode ser instanciada.	Não pode ser instanciada.
Possui apenas métodos implementados.	Pode possuir métodos implementados e abstratos.
Não obriga implementação pelas subclasses.	Pode obrigar implementação pelas subclasses.
Representa objetos concretos.	Representa conceitos genéricos.

As classes abstratas são recomendadas quando:

- ✿ Existe um conceito genérico que será especializado;
- ✿ Diversas classes compartilham atributos e métodos;
- ✿ Deseja-se evitar a duplicação de código;
- ✿ É necessário definir comportamentos obrigatórios para as subclasses.

Capítulo 14 Interfaces

Uma **interface** é um recurso da Programação Orientada a Objetos utilizado para definir um conjunto de comportamentos que uma classe deve implementar. Diferentemente das classes, uma interface não é utilizada para armazenar estado dos objetos, mas sim para especificar quais métodos uma classe deverá possuir.

Uma interface funciona como um *contrato*: qualquer classe que implementar a interface será obrigada a fornecer a implementação de todos os métodos definidos nela.

As interfaces são amplamente utilizadas para promover reutilização de código, desacoplamento entre componentes e maior flexibilidade no desenvolvimento de sistemas.

Uma interface é declarada utilizando a palavra-chave `interface`.

Exemplo 14.1

```
1 interface Animal {  
2  
3     void emitirSom();  
4  
5 }
```

Nesse exemplo, a interface `Animal` define o método `emitirSom()`, mas não informa como ele será executado.

Uma classe utiliza a palavra-chave `implements` para implementar uma interface.

Exemplo 14.2

```
1 interface Animal {  
2     void emitirSom();  
3 }  
4  
5 class Cachorro implements Animal {  
6  
7     @Override  
8     public void emitirSom() {  
9         System.out.println("Au Au!");  
10    }  
11 }
```

A classe `Cachorro` é obrigada a implementar o método `emitirSom()`, pois ele foi definido na interface.

14.1 Utilizando Objetos de Classes que Implementam Interfaces

Após implementar a interface, os objetos podem ser criados normalmente.

Exemplo 14.3

```
1 interface Animal {
2     void emitirSom();
3 }
4
5 class Cachorro implements Animal {
6
7     @Override
8     public void emitirSom() {
9         System.out.println("Au Au!");
10    }
11 }
12
13 public class Principal {
14
15     public static void main(String[] args) {
16
17         Cachorro cachorro = new Cachorro();
18
19         cachorro.emitirSom();
20
21    }
22 }
23
24
25 //Saída
26 //Au
27 //Au
```

14.2 Implementando Múltiplas Interfaces

Uma das grandes vantagens das interfaces é permitir que uma classe implemente várias interfaces simultaneamente.

Exemplo 14.4

```
1 interface Nadador {
2     void nadar();
3 }
4
5 interface Corredor {
6     void correr();
7 }
8
9 class Triatleta implements Nadador, Corredor {
10
11     @Override
12     public void nadar() {
13         System.out.println("Nadando...");
14    }
```

```
15
16 @Override
17 public void correr() {
18     System.out.println("Correndo...");
19 }
20 }
```

14.3 Interfaces e Polimorfismo

Interfaces são muito utilizadas em conjunto com o polimorfismo. Uma variável do tipo da interface pode armazenar objetos de qualquer classe que a implemente.

Exemplo 14.5

```
1 interface Animal {
2     void emitirSom();
3 }
4
5 class Cachorro implements Animal {
6
7     @Override
8     public void emitirSom() {
9         System.out.println("Au Au!");
10    }
11 }
12
13 class Gato implements Animal {
14
15     @Override
16     public void emitirSom() {
17         System.out.println("Miau!");
18    }
19 }
20
21 public class Principal {
22
23     public static void main(String[] args) {
24
25         Animal a1 = new Cachorro();
26         Animal a2 = new Gato();
27
28         a1.emitirSom();
29         a2.emitirSom();
30
31    }
32 }
33
34 // Saída
35 // Au Au!
```

```
36 // Miau!
```

14.4 Métodos Default

A partir do Java 8, as interfaces podem conter métodos com implementação utilizando a palavra-chave `default`.

Exemplo 14.6

```
1 interface Veiculo {
2
3     void acelerar();
4
5     default void ligar() {
6         System.out.println(
7             "Veiculo ligado.");
8     }
9 }
```

As classes que implementarem essa interface poderão utilizar o método `ligar()` sem precisar reescrevê-lo.

14.5 Métodos Estáticos em Interfaces

Interfaces também podem possuir métodos estáticos.

Exemplo 14.7

```
1 interface Calculadora {
2
3     static int somar(int a, int b) {
4         return a + b;
5     }
6 }
7
8 public class Principal {
9
10     public static void main(String[] args) {
11
12         int resultado = Calculadora.somar(10, 20);
13
14     }
15
16 }
```

14.6 Diferenças entre Herança e Interface

Tabela 14.1. Comparação entre Herança e Interface

Característica	Classe	Interface
Utiliza	extends	implements
Herança múltipla	Não	Sim
Possui construtor	Sim	Não
Possui atributos de instância	Sim	Não
Pode conter métodos abstratos	Sim	Sim
Pode conter métodos com implementação	Sim	Sim (default e static)
Objetivo principal	Reutilização de código	Definir contratos

Capítulo 15 Array Unidimensional

Um array (ou vetor) é uma estrutura de dados que armazena vários valores do mesmo tipo em uma única variável, organizados de forma sequencial na memória. O array é composto por diferentes elementos, entre eles:

- ❁ **Tipo de dado**, todos os valores devem ser do mesmo tipo, caracterizando uma estrutura homogênea.
- ❁ **Tamanho**, representa a quantidade total de elementos que o array pode armazenar, tem um tamanho definido na criação e não pode ser alterado.
- ❁ **Índices (posições)**, São índices que indicam a localização de cada elemento dentro do array, sempre iniciando em zero.

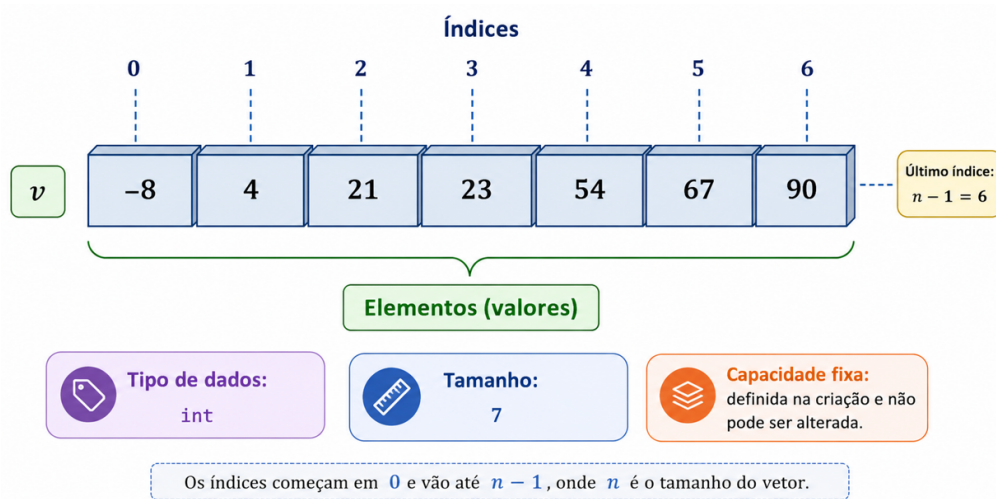


Figura 15.1. Estrutura de um vetor (array) unidimensional com índices e elementos.

15.1 Criação

A criação de um array envolve a definição do tipo de dado, do nome da variável e do número de posições que ele irá armazenar. Existem duas formas principais de criar um array: definindo o tamanho do array diretamente ou informando os valores no momento da criação.

Exemplo 15.1

```
1 public class ExemploArrayPratico {
2     public static void main(String[] args) {
3
4         // Declaração e criação de um array de inteiros com 5 posições
5         int[] numeros = new int[5];
6
7         // Atribuição de valores às posições do array
8         numeros[0] = 10;
9         numeros[1] = 20;
10        numeros[2] = 30;
11        numeros[3] = 40;
```

```
12     numeros[4] = 50;
13
14     // Exibição de alguns elementos
15     System.out.println(numeros[0]); // Seleciona o índice (posição) 0 e exibe 10
16
17     // Criação de um array já inicializado com valores
18     String[] nomes = {"Ana", "Bruno", "Carlos"};
19
20     // Exibição de alguns elementos
21     System.out.println(nomes[1]); // Seleciona o índice (posição) 1 e exibe
Bruno
22 }
23 }
```

15.2 Tipos

Em Java, os arrays com tipos de dados primitivos podem utilizar as seguintes variáveis: `int`, `double`, `float`, `char`, `boolean`, `byte`, `short` e `long`.

Exemplo 15.2

```
1 public class ArraysPrimitivos {
2     public static void main(String[] args) {
3
4         // int (números inteiros)
5         int[] inteiros = {10, 20, 30};
6
7         // double (números decimais de maior precisão)
8         double[] decimais = {2.5, 3.14, 9.81};
9
10        // float (números decimais de menor precisão)
11        float[] reais = {1.5f, 2.5f, 3.5f};
12
13        // char (caracteres)
14        char[] letras = {'A', 'B', 'C'};
15
16        // boolean (valores lógicos)
17        boolean[] valoresLogicos = {true, false, true};
18
19        // byte (inteiros pequenos)
20        byte[] pequenos = {1, 2, 3};
21
22        // short (inteiros de tamanho intermediário)
23        short[] medios = {1000, 2000, 3000};
24
25        // long (inteiros grandes)
26        long[] grandes = {100000L, 200000L, 300000L};
27
28        // Exemplo de acesso
```

```
29     System.out.println(inteiros[0]); // 10
30     System.out.println(decimais[1]); // 3.14
31     System.out.println(letras[2]);    // C
32 }
33 }
```

Arrays podem armazenar tipos por referência, como *String*, objetos e outras estruturas.

Exemplo 15.3

```
1 public class ArrayObjeto {
2     public static void main(String[] args) {
3
4         // Declaração e inicialização do array
5         String[] nomes = {"Pedro", "Maria", "Juca"};
6
7         // Percorrendo o array com for
8         for (int i = 0; i < nomes.length; i++) {
9             System.out.println("Posição " + i + ": " + nomes[i]);
10        }
11    }
12 }
13 }
```

15.3 Acesso

O acesso a um elemento é feito por meio de índices, que indicam a posição de cada elemento. Em Java, os índices sempre começam em zero.

Exemplo 15.4

```
1 public class AcessoArray {
2     public static void main(String[] args) {
3
4         int[] numeros = {10, 20, 30, 40};
5
6         // Acessando elementos
7         System.out.println(numeros[0]); // 10
8         System.out.println(numeros[2]); // 30
9     }
10 }
```

Atenção: acessar uma posição inválida do array gera um erro em tempo de execução (`ArrayIndexOutOfBoundsException`). O atributo `length` indica o número de elementos do array.

15.4 Pesquisa

A pesquisa consiste em procurar um valor dentro do array, normalmente utilizando um laço de repetição.

Exemplo 15.5

```
1 public class PesquisaArray {
2     public static void main(String[] args) {
3
4         int[] numeros = {10, 20, 30, 40};
5         int busca = 30;
6
7         for (int i = 0; i < numeros.length; i++) {
8             if (numeros[i] == busca) {
9                 System.out.println("Encontrado na posição: " + i);
10            }
11        }
12    }
13 }
```

15.5 Percorrer com for-each

O laço for-each é uma forma simplificada de percorrer arrays, permitindo acessar diretamente cada elemento, sem a necessidade de utilizar índices. Esse tipo de laço é especialmente útil quando desejamos apenas ler os valores do array.

No entanto, o for-each não permite acessar o índice dos elementos nem alterar diretamente suas posições.

Exemplo 15.6

```
1 public class PercorrerArrayForEach {
2     public static void main(String[] args) {
3
4         // Declaração e inicialização do array
5         String[] carros = {"BYD", "BMW", "Ford", "Hyundai"};
6
7         // Percorrendo o array com for-each
8         for (String carro : carros) {
9             System.out.println("Carro: " + carro);
10        }
11
12        // Saída esperada:
13        // Carro: BYD
14        // Carro: BMW
15        // Carro: Ford
16        // Carro: Hyundai
17    }
18 }
```

15.6 Alteração

Os elementos de um array podem ser modificados acessando diretamente uma posição específica (índice) e atribuindo a ela um novo valor.

Exemplo 15.7

```
1 public class AlteracaoArray {
2     public static void main(String[] args) {
3
4         // Declaração e inicialização do array
5         int[] numeros = {10, 20, 30, 40};
6
7         // Alterando o valor da posição 1 (segundo elemento)
8         numeros[1] = 50;
9
10        // Percorrendo o array para exibir os valores atualizados
11        for (int i = 0; i < numeros.length; i++) {
12            System.out.println(numeros[i]);
13        }
14
15        // Resultado esperado:
16        // {10, 50, 30, 40}
17    }
18 }
```

15.7 Remoção (limitações)

Arrays possuem tamanho fixo e não permitem a remoção direta de elementos. Uma alternativa é substituir o valor da posição desejada por um valor padrão. Por exemplo, para tipos primitivos, como `int`, é comum utilizar o valor zero ou 9999. Já para tipos por referência, como `String`, pode-se utilizar o valor `null` para indicar ausência de objeto.

Exemplo 15.8

```
1 public class RemocaoArray {
2     public static void main(String[] args) {
3
4         // Array de tipo primitivo (int)
5         int[] numeros = {10, 20, 30, 40};
6
7         // Substituindo o valor da posição 1 por 0
8         numeros[1] = 0;
9
10        // Array de tipo por referência (String)
11        String[] nomes = {"Ana", "Bruno", "Carlos"};
12
13        // Substituindo a referência da posição 1 por null
14        nomes[1] = null;
15    }
```

```
16 // Exibindo os arrays
17 System.out.println("Array de inteiros:");
18 for (int i = 0; i < numeros.length; i++) {
19     System.out.println(numeros[i]);
20 }
21
22 System.out.println("Array de Strings:");
23 for (int i = 0; i < nomes.length; i++) {
24     System.out.println(nomes[i]);
25 }
26
27 // Resultado esperado:
28 // numeros -> {10, 0, 30, 40}
29 // nomes   -> {"Ana", null, "Carlos"}
30 }
31 }
```

15.8 Exercícios Resolvidos

Para percorrer um array e realizar operações com seus elementos, utilizamos estruturas de repetição, como o laço for. Neste exemplo, somamos todos os valores do array.

Exercício Resolvidos 15.1

```
1 public class SomaArray {
2     public static void main(String[] args) {
3
4         int[] numbers = {1, 5, 10, 25};
5         int sum = 0;
6
7         // Percorrendo o array e somando os elementos
8         for (int i = 0; i < numbers.length; i++) {
9             sum += numbers[i];
10        }
11
12        System.out.println("A soma é: " + sum);
13    }
14 }
```

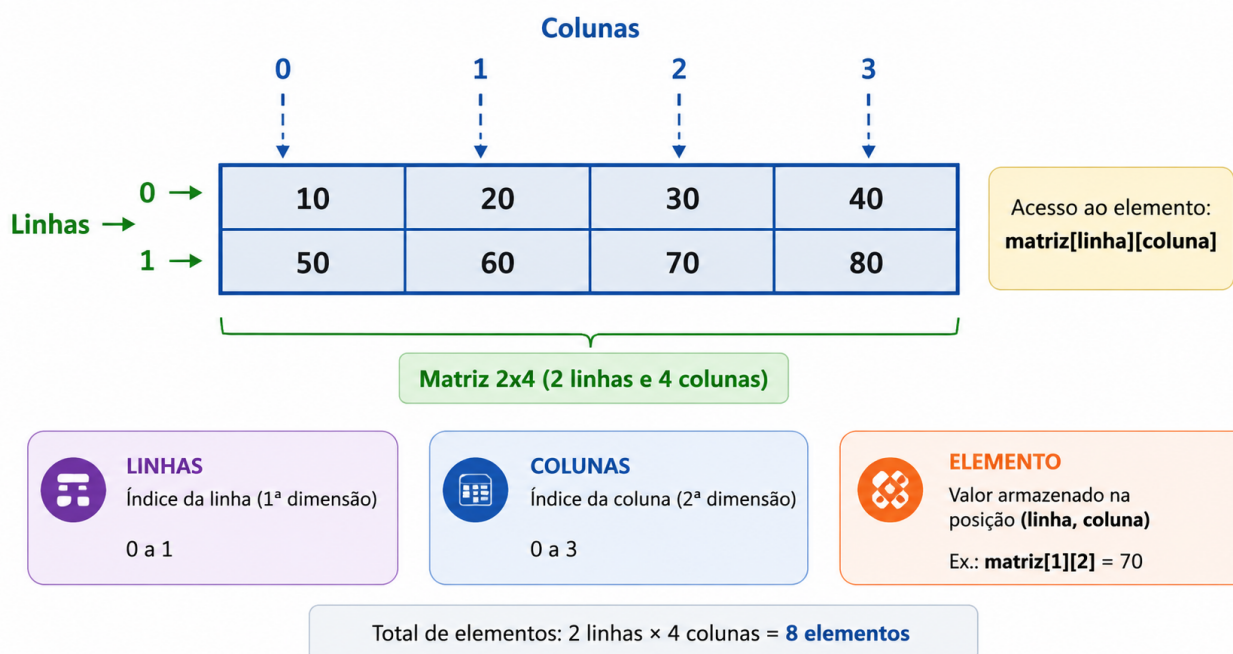
Capítulo 16 Arrays Multidimensionais

Um array multidimensional é uma estrutura formada por um array dentro de outros arrays, permitindo armazenar dados organizados em múltiplas dimensões, como linhas e colunas.

Esse tipo de estrutura é amplamente utilizado para representar matrizes, tabelas, grades e outros conjuntos de dados organizados de forma tabular. Os arrays multidimensionais facilitam o armazenamento, a organização e o gerenciamento de grandes quantidades de dados relacionados entre si.

Neste curso, iremos tratar somente o array bidimensional (2D), no qual. A imagem abaixo apresenta o esquema de um array bidimensional.

Figura 16.1. Array bidimensional com 3 linhas e 3 colunas.



Na imagem acima, o índice das linhas varia de 0 até $(x - 1)$, enquanto o índice das colunas varia de 0 até $(y - 1)$. Onde, x representa o número total de linhas e y representa o número total de colunas.

👉 Observação

Os índices sempre começam em 0, o último índice sempre será uma unidade menor que a quantidade total de elementos daquela dimensão.

16.1 Criando um Array Bidimensional

Existem diferentes formas de declarar e inicializar arrays bidimensionais em Java. Uma das maneiras mais comuns consiste em declarar a matriz com tamanho definido e, posteriormente, atribuir valores manualmente para cada posição.

Nesse tipo de declaração, primeiro é criada a estrutura da matriz informando a quantidade de linhas e colunas. Em seguida, os elementos são preenchidos individualmente por meio de seus índices.

Exemplo 16.1

```
1 public class CriarArrayBidimensional {
2     public static void main(String[] args) {
3         // Declarando o array bidimensional
4         // Criando a matriz com 1 linha e 3 colunas
5         int[][] arr = new int[1][3];
6         // Atribuindo valores
7         arr[0][0] = 3;
8         arr[0][1] = 5;
9         arr[0][2] = 7;
10        // Exibindo os valores
11        System.out.println("arr[0][0] = " + arr[0][0]);
12        System.out.println("arr[0][1] = " + arr[0][1]);
13        System.out.println("arr[0][2] = " + arr[0][2]);
14        //arr[0][0] = 3
15        //arr[0][1] = 5
16        //arr[0][2] = 7
17    }
18 }
```

Outro exemplo de criação de array bidimensional ocorre quando os valores são adicionados diretamente durante a declaração da matriz. Nesse tipo de inicialização, não é necessário informar explicitamente a quantidade de linhas e colunas, pois o Java determina automaticamente o tamanho da matriz com base nos valores fornecidos.

Esse modelo é bastante utilizado quando os dados da matriz já são conhecidos no momento da criação.

Exemplo 16.2

```
1 public class PercorrerArrayBidimensional {
2     public static void main(String[] args) {
3         // Array bidimensional inicializado
4         int[][] arr = {
5             {1, 2},
6             {3, 4}
7         };
8         // Percorrendo as linhas
9         for (int i = 0; i < 2; i++) {
10            // Percorrendo as colunas
11            for (int j = 0; j < 2; j++) {
12                System.out.print(arr[i][j] + " ");
13            }
14            // Quebra de linha
15            System.out.println();
16        }
17        // Saída:
18        //1 2
19        //3 4
20    }
21 }
```

16.2 Acessando Elementos de um Array Bidimensional

Em um array bidimensional, cada elemento é identificado por dois índices:

- ❖ o primeiro índice representa a **linha**;
- ❖ o segundo índice representa a **coluna**.

O acesso aos elementos é realizado utilizando a seguinte estrutura: `array[linha][coluna]`. Os arrays utilizam indexação iniciada em 0. Isso significa que:

- ❖ o primeiro elemento da linha possui índice 0;
- ❖ a primeira coluna também possui índice 0.

Assim:

- ❖ `array[0][0]` representa o elemento localizado na primeira linha e primeira coluna;
- ❖ `array[1][0]` representa o elemento localizado na segunda linha e primeira coluna;
- ❖ `array[1][1]` representa o elemento localizado na segunda linha e segunda coluna.

O exemplo a seguir cria uma matriz 2x2 e acessa um elemento específico utilizando seus índices.

Exemplo 16.3

```

1 public class AcessarElementos {
2     public static void main(String[] args) {
3
4         // Criando um array bidimensional
5         int[][] arr = {
6             {1, 2},
7             {3, 4}
8         };
9
10        // Acessando o elemento da linha 1
11        // e coluna 1
12        System.out.println("arr[1][1] = " + arr[1][1]);
13    }
14    // Saída:
15    //arr[1][1] = 4
16 }
```

16.3 Removendo Elementos de um Array Bidimensional

Em Java, não é possível remover diretamente um elemento de um array bidimensional, pois os arrays possuem tamanho fixo. Após a criação da matriz, a quantidade de linhas e colunas não pode ser alterada. Isso significa que:

- ❖ Não é possível apagar fisicamente uma posição do array;
- ❖ Não é possível diminuir automaticamente o tamanho da matriz;
- ❖ Todas as posições continuam existindo durante a execução do programa.

Por esse motivo, a remoção normalmente é realizada de forma lógica, substituindo o valor armaze-

nado ou criando uma nova matriz. A remoção lógica consiste em substituir o valor do elemento por um valor padrão. O exemplo a seguir substitui o valor 5 por 99999.

Exemplo 16.4

```
1 public class RemoverElemento {
2     public static void main(String[] args) {
3
4         int[][] numeros = {
5             {1, 2, 3},
6             {4, 5, 6}
7         };
8
9         // Removendo logicamente o elemento
10        numeros[1][1] = 99999;
11
12        // Exibindo a matriz
13        for (int i = 0; i < numeros.length; i++) {
14
15            for (int j = 0; j < numeros[i].length; j++) {
16
17                System.out.print(numeros[i][j] + " ");
18            }
19
20            System.out.println();
21        }
22    }
23    // Saída:
24    //1 2 3
25    //4 99999 6
26
27 }
```

** Observação**

Outra alternativa consiste em criar uma nova matriz sem o elemento desejado. Essa abordagem exige: (i) criar uma nova estrutura; (ii) copiar os elementos desejados; (iii) reorganizar as posições da matriz.

Essa técnica é mais trabalhosa e normalmente utilizada em situações específicas.

16.4 Alterando o Valor de um Elemento

Os elementos de um array bidimensional podem ter seus valores alterados a qualquer momento durante a execução do programa. Para modificar um elemento, basta acessar sua posição utilizando os índices da linha e da coluna e atribuir um novo valor.

O exemplo a seguir altera o valor armazenado na linha 1 e coluna 1 da matriz.

Exemplo 16.5

```
1 public class AlterarElemento {
2
3     public static void main(String[] args) {
4
5         int[][] numeros = {
6             {1, 2, 3},
7             {4, 5, 6}
8         };
9
10        // Exibindo valor original
11        System.out.println("Valor original:");
12        System.out.println(numeros[1][1]);
13
14        // Alterando o elemento
15        numeros[1][1] = 99;
16
17        // Exibindo novo valor
18        System.out.println("Novo valor:");
19        System.out.println(numeros[1][1]);
20    }
21
22    // Saída:
23    //Valor original:
24    //5
25    //Novo valor:
26    //99
27
28 }
```

16.5 Percorrendo um Array Multidimensional

Percorrer um array multidimensional significa acessar todos os seus elementos de forma sequencial. Em arrays bidimensionais, isso normalmente é realizado utilizando estruturas de repetição aninhadas. Nesse processo:

- ❁ O primeiro laço percorre as linhas;
- ❁ O segundo laço percorre as colunas;
- ❁ Cada elemento da matriz é acessado individualmente.

A estrutura mais comum para percorrer um array bidimensional utiliza dois laços `for` usando variáveis `i` e `j`. Onde:

- ❁ `i` representa o índice das linhas;
- ❁ `j` representa o índice das colunas;
- ❁ `matriz[i][j]` acessa cada elemento da matriz.

O exemplo a seguir percorre uma matriz 3x3 e imprime todos os seus elementos.

Exemplo 16.6

```
1 public class PercorrerMatriz {
2     public static void main(String[] args) {
3
4         int[][] numeros = {
5             {1, 2, 3},
6             {4, 5, 6},
7             {7, 8, 9}
8         };
9
10        // Percorrendo as linhas
11        for (int i = 0; i < numeros.length; i++) {
12
13            // Percorrendo as colunas
14            for (int j = 0; j < numeros[i].length; j++) {
15
16                System.out.print(
17                    numeros[i][j] + " "
18                );
19            }
20
21            // Quebra de linha
22            System.out.println();
23        }
24    }
25    // Saída:
26    // 1 2 3
27    // 4 5 6
28    // 7 8 9
29 }
```

Também é possível percorrer arrays multidimensionais utilizando o laço for-each.

Exemplo 16.7

```
1 public class PercorrerForEach {
2     public static void main(String[] args) {
3         int[][] numeros = {
4             {1, 2, 3},
5             {4, 5, 6}
6         };
7         for (int[] linha : numeros) {
8             for (int valor : linha) {
9                 System.out.print(valor + " ");
10            }
11            System.out.println();
12        }
13    }
14    // 1 2 3
15    // 4 5 6
16 }
```

16.6 Pesquisando Elementos em um Array Bidimensional

Pesquisar em um array bidimensional significa verificar se um determinado valor existe dentro da matriz. Para isso, normalmente utilizamos estruturas de repetição aninhadas, percorrendo todas as linhas e colunas do array. Durante a pesquisa:

- ❁ O primeiro laço percorre as linhas;
- ❁ O segundo laço percorre as colunas;
- ❁ Cada elemento é comparado com o valor procurado.

O exemplo a seguir procura o valor 50 dentro de uma matriz.

Exemplo 16.8

```

1 public class PesquisarElemento {
2     public static void main(String[] args) {
3         int[][] numeros = {
4             {10, 20, 30},
5             {40, 50, 60},
6             {70, 80, 90}
7         };
8         int valorProcurado = 50;
9         boolean encontrado = false;
10        // Percorrendo as linhas
11        for (int i = 0; i < numeros.length; i++) {
12            // Percorrendo as colunas
13            for (int j = 0; j < numeros[i].length; j++) {
14                // Comparando o elemento
15                if (numeros[i][j] == valorProcurado) {
16                    System.out.println(
17                        "Valor encontrado na linha "
18                        + i +
19                        " e coluna "
20                        + j
21                    );
22
23                    encontrado = true;
24                }
25            }
26        }
27        // Caso nao encontre o valor
28        if (!encontrado) {
29            System.out.println("Valor nao encontrado.");
30        }
31    }
32    // Saida:
33    //Valor encontrado na linha 1 e coluna 1
34 }

```


Capítulo 17 ArrayList

Implementa uma lista dinâmica baseada em arrays. Diferentemente dos arrays tradicionais, que possuem tamanho fixo definido no momento da criação, o ArrayList pode aumentar ou reduzir automaticamente sua capacidade conforme a necessidade da aplicação.

Essa característica torna a estrutura mais flexível para manipulação de dados em tempo de execução, permitindo adicionar, remover e acessar elementos de forma prática e eficiente. A implementação, envolve a biblioteca `java.util.` e as principais características do ArrayList.

- ✿ Estrutura implementada internamente por meio de um array
- ✿ Tamanho dinâmico, ajustado automaticamente
- ✿ Permite armazenar elementos duplicados
- ✿ Mantém a ordem de inserção dos elementos
- ✿ Possui acesso rápido aos elementos por índice
- ✿ Inserções e remoções em posições intermediárias podem exigir deslocamento de elementos, aumentando o custo da operação

Exemplo 17.1

```
1 import java.util.ArrayList;
2
3 public class ExemploArrayList {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         ArrayList<String> nomes = new ArrayList<>();
8
9         // Adicionando elementos
10        nomes.add("Ana");
11        nomes.add("Carlos");
12        nomes.add("Maria");
13
14        // Exibindo elementos
15        System.out.println("Lista de nomes:");
16        for (String nome : nomes) {
17            System.out.println(nome);
18        }
19    }
20 }
```

O ArrayList pode ser criado com uma capacidade inicial definida pelo programador, porém isso não significa que seu tamanho ficará fixo. A capacidade inicial apenas determina quantas posições internas serão reservadas inicialmente na memória, reduzindo a necessidade de redimensionamentos imediatos.

Mesmo quando uma capacidade inicial é especificada, o ArrayList continua sendo dinâmico e pode crescer automaticamente conforme novos elementos são adicionados.

Exemplo 17.2

```

1 import java.util.ArrayList;
2 public class ExemploArrayListNumeros {
3     public static void main(String[] args) {
4
5         // Declaração e criação com capacidade inicial
6         ArrayList<Integer> numeros = new ArrayList<>(4);
7         // Adicionando elementos
8         numeros.add(10);
9         numeros.add(20);
10        numeros.add(30);
11        numeros.add(40);
12
13        // Exibindo os elementos
14        System.out.println("Números na lista:");
15        for (int num : numeros) {
16            System.out.println(num);
17        }
18        // Exibindo o tamanho atual da lista
19        System.out.println("Quantidade de elementos: " + numeros.size());
20    }
21 }

```

17.1 Métodos

Os principais métodos da classe `ArrayList` permitem manipular elementos de forma simples e eficiente, incluindo operações de inserção, remoção, acesso e verificação de dados.

Tabela 17.1. Principais métodos da classe `ArrayList`

Método	Descrição
<code>add(E e)</code>	Adiciona um elemento ao final da lista.
<code>add(int index, E e)</code>	Insere um elemento em uma posição específica.
<code>get(int index)</code>	Retorna o elemento da posição informada.
<code>set(int index, E e)</code>	Substitui o elemento em uma posição específica.
<code>remove(int index)</code>	Remove o elemento pelo índice.
<code>remove(Object o)</code>	Remove a primeira ocorrência do elemento informado.
<code>size()</code>	Retorna a quantidade de elementos na lista.
<code>clear()</code>	Remove todos os elementos da lista.
<code>contains(Object o)</code>	Verifica se o elemento existe na lista.
<code>equals(Object o)</code>	Compara duas listas, verificando se possuem os mesmos elementos e na mesma ordem.
<code>isEmpty()</code>	Verifica se a lista está vazia.

Exemplo 17.3

```
1 import java.util.ArrayList;
2
3 public class ExemploMetodosArrayList {
4
5     public static void main(String[] args) {
6
7         // Declaração e criação
8         ArrayList<String> nomes = new ArrayList<>();
9
10        // Adicionando elementos
11        nomes.add("Ana");
12        nomes.add("Carlos");
13        nomes.add("Maria");
14
15        // get: acessar elemento
16        System.out.println("Primeiro nome: " + nomes.get(0));
17
18        // set: alterar elemento
19        nomes.set(1, "João");
20
21        // contains: verificar existência
22        if (nomes.contains("Ana")) {
23            System.out.println("Ana está na lista");
24        }
25
26        // remove: remover elemento
27        nomes.remove("Maria");
28
29        // size: quantidade de elementos
30        System.out.println("Tamanho da lista: " + nomes.size());
31
32        // Exibindo lista atual
33        System.out.println("Lista: " + nomes);
34
35        // clear: remover todos os elementos
36        nomes.clear();
37
38        // isEmpty: verificar se está vazia
39        if (nomes.isEmpty()) {
40            System.out.println("A lista está vazia");
41        }
42    }
43 }
44
45 }
```

Exercício Resolvidos 17.1

1 Crie um programa que armazene 10 números inteiros em um ArrayList. Em seguida, percorra a lista e mostre apenas os números pares.}

```
2
3 import java.util.ArrayList;
4
5 public class QuestaoUmArrayList {
6     public static void main(String[] args) {
7
8         ArrayList<Integer> numeros = new ArrayList<>();
9
10        numeros.add(10);
11        numeros.add(15);
12        numeros.add(8);
13        numeros.add(21);
14        numeros.add(30);
15        numeros.add(7);
16        numeros.add(18);
17        numeros.add(5);
18        numeros.add(12);
19        numeros.add(9);
20
21        System.out.println("Numeros pares:");
22
23        for (int i = 0; i < numeros.size(); i++) {
24
25            if (numeros.get(i) % 2 == 0) {
26                System.out.println(numeros.get(i));
27            }
28        }
29    }
```

Exercício Resolvidos 17.2

1 Desenvolva um programa que armazene valores inteiros em um ArrayList e conte quantos números são maiores que 50.

```
2
3 import java.util.ArrayList;
4
5 public class QuestaoDoisArrayList {
6     public static void main(String[] args) {
7
8         ArrayList<Integer> valores = new ArrayList<>();
9
10        valores.add(25);
11        valores.add(80);
12        valores.add(60);
13        valores.add(10);
14        valores.add(95);
15        valores.add(40);
```

```
16 valores.add(70);
17
18 int contador = 0;
19
20 for (int i = 0; i < valores.size(); i++) {
21     if (valores.get(i) > 50) {
22         contador++;
23     }
24 }
25
26
27 System.out.println(
28     "Quantidade de numeros maiores que 50: "
29     + contador
30 );
31 }
```

Exercício Resolvidos 17.3

1 Crie um programa que armazene nomes em um `ArrayList`. Utilize uma estrutura de repetição para remover todos os nomes com menos de 5 caracteres.

```
2
3 import java.util.ArrayList;
4 public class QuestaoTresArrayList {
5     public static void main(String[] args) {
6
7         ArrayList<Integer> valores = new ArrayList<>();
8         valores.add(25);
9         valores.add(80);
10        valores.add(60);
11        valores.add(10);
12        valores.add(95);
13        valores.add(40);
14        valores.add(70);
15        int contador = 0;
16        for (int i = 0; i < valores.size(); i++) {
17
18            if (valores.get(i) > 50) {
19                contador++;
20            }
21        }
22        System.out.println(
23            "Quantidade de numeros maiores que 50: "
24            + contador
25        );
26    }
```

Exercício 17.1

ListaAleatoriaCondicional - Gere um ArrayList com 100 números aleatórios entre 1 e 100. Verifique o valor do penúltimo elemento da lista. Caso o penúltimo número seja maior que 50, adicione o número 33 na posição 51. Caso contrário, adicione o número 99 na posição 51. Ao final, exiba todos os elementos da lista.

Exercício 17.2

ListaAleatoriaCondicionalMetodoRemocao - Com base na questão anterior, implemente um método que remova todos os elementos duplicados da lista. Testar exibindo a lista final sem valores repetidos.

Exercício 17.3

ListaAleatoriaContarRepetidos - Modifique o programa anterior adicionando um método capaz de contar quantas vezes cada elemento repetido aparece no ArrayList.

Capítulo 18 HashSet

O HashSet é uma classe da biblioteca `java.util` que implementa uma coleção baseada em tabela hash. Ele é utilizado para armazenar elementos de forma eficiente, sem permitir valores duplicados.

Diferente do `ArrayList`, o HashSet não mantém a ordem de inserção dos elementos.

18.1 Características

- ❖ Não permite elementos duplicados
- ❖ Não mantém ordem dos elementos
- ❖ Permite valores nulos (apenas um)
- ❖ Alta performance para inserção, remoção e busca

Exemplo 18.1

```
1 import java.util.HashSet;
2
3 public class ExemploHashSetCarros {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         HashSet<String> carros = new HashSet<>();
8
9         // Adicionando elementos
10        carros.add("Volvo");
11        carros.add("BMW");
12        carros.add("Ford");
13        carros.add("BMW"); // elemento duplicado (não será inserido)
14        carros.add("Mazda");
15
16        // Exibindo os elementos
17        System.out.println("Carros cadastrados:");
18        System.out.println(carros);
19
20        // Exibindo quantidade de elementos
21        System.out.println("Quantidade: " + carros.size());
22    }
23 }
```

18.2 Métodos

Os principais métodos da classe HashSet permitem manipular elementos de forma simples e eficiente, incluindo operações de inserção, remoção, acesso e verificação de dados.

Tabela 18.1. Principais métodos da classe HashSet

Método	Descrição
add(E e)	Adiciona um elemento ao conjunto.
remove(Object o)	Remove o elemento informado.
contains(Object o)	Verifica se o elemento existe.
size()	Retorna a quantidade de elementos.
clear()	Remove todos os elementos.
isEmpty()	Verifica se o conjunto está vazio.

Exemplo 18.2

```

1 import java.util.HashSet;
2 public class ExemploHashSet {
3     public static void main(String[] args) {
4
5         // Declaração e criação
6         HashSet<String> nomes = new HashSet<>();
7
8         // Adicionando elementos
9         nomes.add("Ana");
10        nomes.add("Carlos");
11        nomes.add("Ana"); // duplicado (não será adicionado)
12
13        // Exibindo elementos
14        System.out.println("Elementos: " + nomes);
15
16        // Verificando elemento
17        if (nomes.contains("Carlos")) {
18            System.out.println("Carlos está no conjunto");
19        }
20
21        // Tamanho
22        System.out.println("Quantidade: " + nomes.size());
23
24        // Removendo elemento
25        nomes.remove("Ana");
26
27        // Verificando vazio
28        if (nomes.isEmpty()) {
29            System.out.println("Conjunto vazio");
30        } else {
31            System.out.println("Ainda possui elementos");
32        }
33    }
34 }

```

Capítulo 19 HashMap

O HashMap é uma classe da biblioteca `java.util` utilizada para armazenar dados no formato **chave-valor**. Cada chave é única e está associada a um valor.

Essa estrutura é muito utilizada quando precisamos realizar buscas rápidas com base em uma chave.

19.1 Características

- ✿ Armazena dados no formato chave - valor
- ✿ Não permite chaves duplicadas
- ✿ Permite valores duplicados
- ✿ Não mantém ordem dos elementos
- ✿ Alta eficiência para busca e inserção

Exemplo 19.1

```
1 import java.util.HashMap;
2
3 public class ExemploHashMapPaises {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         HashMap<String, String> capitais = new HashMap<>();
8
9         // Adicionando elementos (País, Capital)
10        capitais.put("Inglaterra", "Londres");
11        capitais.put("Índia", "Nova Délhi");
12        capitais.put("Áustria", "Viena");
13        capitais.put("Noruega", "Oslo");
14        capitais.put("Noruega", "Oslo"); // chave duplicada (não será adicionada
15        novamente)
16        capitais.put("EUA", "Washington DC");
17
18        // Exibindo os dados
19        System.out.println("Capitais cadastradas:");
20        System.out.println(capitais);
21
22        // Exibindo quantidade de elementos
23        System.out.println("Quantidade: " + capitais.size());
24    }
```

19.2 Métodos

Os principais métodos da classe HashMap permitem manipular elementos de forma simples e eficiente, incluindo operações de inserção, remoção, acesso e verificação de dados.

Tabela 19.1. Principais métodos da classe HashMap

Método	Descrição
put(K key, V value)	Adiciona ou atualiza um par chave-valor.
get(Object key)	Retorna o valor associado à chave.
remove(Object key)	Remove o elemento pela chave.
containsKey(Object key)	Verifica se a chave existe.
containsValue(Object value)	Verifica se o valor existe.
size()	Retorna a quantidade de elementos.
clear()	Remove todos os elementos.
isEmpty()	Verifica se o mapa está vazio.

Exemplo 19.2

```

1 import java.util.HashMap;
2
3 public class ExemploHashMap {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         HashMap<String, Integer> alunos = new HashMap<>();
8
9         // Adicionando elementos (chave, valor)
10        alunos.put("Ana", 8);
11        alunos.put("Carlos", 9);
12        alunos.put("Maria", 7);
13
14        // Acessando valor pela chave
15        System.out.println("Nota de Ana: " + alunos.get("Ana"));
16
17        // Verificando chave
18        if (alunos.containsKey("Carlos")) {
19            System.out.println("Carlos está cadastrado");
20        }
21
22        // Exibindo quantidade
23        System.out.println("Quantidade: " + alunos.size());
24
25        // Removendo elemento
26        alunos.remove("Maria");
27
28        // Exibindo estrutura
29        System.out.println(alunos);
30    }
31 }

```

Capítulo 20 Fila

A fila (Queue) é uma interface da biblioteca `java.util` utilizada para armazenar elementos seguindo a lógica **FIFO** (First In, First Out), ou seja, o primeiro elemento inserido é o primeiro a ser removido.

Ela é muito utilizada em situações como filas de atendimento, processamento de tarefas e controle de execução.

20.1 Características

- ✿ Estrutura do tipo fila (FIFO)
- ✿ Permite inserção no final e remoção no início
- ✿ Não permite acesso direto por índice
- ✿ Possui diferentes implementações (ex: `LinkedList`, `PriorityQueue`)

Como fila é uma interface, utilizamos uma classe que a implementa, como `LinkedList`:

Exemplo 20.1

```
1 import java.util.Queue;
2 import java.util.LinkedList;
3
4 public class ExemploQueue {
5     public static void main(String[] args) {
6
7         // Declaração e criação
8         Queue<String> fila = new LinkedList<>();
9
10        // Adicionando elementos
11        fila.add("Ana");
12        fila.add("Carlos");
13        fila.add("Maria");
14
15        // Exibindo fila
16        System.out.println("Fila: " + fila);
17
18        // Consultando o primeiro elemento
19        System.out.println("Primeiro da fila: " + fila.peek());
20
21        // Removendo elemento
22        fila.poll();
23
24        // Exibindo após remoção
25        System.out.println("Fila atualizada: " + fila);
26    }
27 }
```

20.2 Métodos

Os principais métodos da fila permitem manipular elementos de forma simples e eficiente, incluindo operações de inserção, remoção, acesso e verificação de dados.

Tabela 20.1. Principais métodos da interface Queue

Método	Descrição
<code>add(E e)</code>	Adiciona elemento ao final da fila.
<code>offer(E e)</code>	Adiciona elemento (sem lançar erro se falhar).
<code>remove()</code>	Remove e retorna o primeiro elemento.
<code>poll()</code>	Remove e retorna o primeiro elemento (ou null).
<code>element()</code>	Retorna o primeiro elemento sem remover.
<code>peek()</code>	Retorna o primeiro elemento (ou null).

20.3 Exemplo

Exemplo 20.2

```
1 import java.util.Queue;
2 import java.util.LinkedList;
3
4 public class ExemploQueue {
5     public static void main(String[] args) {
6
7         // Declaração e criação
8         Queue<String> fila = new LinkedList<>();
9
10        // Adicionando elementos
11        fila.add("Ana");
12        fila.add("Carlos");
13        fila.add("Maria");
14
15        // Exibindo fila
16        System.out.println("Fila: " + fila);
17
18        // Consultando o primeiro elemento
19        System.out.println("Primeiro: " + fila.peek());
20
21        // Removendo elemento
22        fila.poll();
23
24        // Exibindo após remoção
25        System.out.println("Fila atualizada: " + fila);
26    }
27 }
```

Capítulo 21 Stack

A pilha, Stack é uma classe da biblioteca `java.util` que representa uma estrutura de dados baseada no princípio **LIFO** (Last In, First Out), ou seja, o último elemento inserido é o primeiro a ser removido.

Essa estrutura é muito utilizada em situações como controle de chamadas de funções, desfazer ações (undo) e processamento de expressões.

21.1 Características

- ✿ Estrutura do tipo pilha (LIFO)
- ✿ Inserção e remoção no topo da pilha
- ✿ Permite acesso ao elemento do topo
- ✿ Baseada na classe `Vector`

Exemplo 21.1

```
1 import java.util.Stack;
2
3 public class ExemploStack {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         Stack<String> pilha = new Stack<>();
8
9         // Adicionando elementos
10        pilha.push("Ana");
11        pilha.push("Carlos");
12        pilha.push("Maria");
13
14        // Exibindo pilha
15        System.out.println("Pilha: " + pilha);
16
17        // Consultando o topo
18        System.out.println("Topo da pilha: " + pilha.peek());
19
20        // Removendo elemento do topo
21        pilha.pop();
22
23        // Exibindo após remoção
24        System.out.println("Pilha atualizada: " + pilha);
25    }
26 }
```

21.2 Métodos

Os principais métodos da classe pilha `Stack` permitem manipular os elementos da pilha, incluindo operações de inserção, remoção e consulta do topo.

Tabela 21.1. Principais métodos da classe Stack

Método	Descrição
push(E item)	Adiciona elemento ao topo da pilha.
pop()	Remove e retorna o elemento do topo.
peek()	Retorna o elemento do topo sem remover.
empty()	Verifica se a pilha está vazia.
search(Object o)	Retorna a posição do elemento na pilha.

Exemplo 21.2

```
1 import java.util.Stack;
2
3 public class ExemploStack {
4     public static void main(String[] args) {
5
6         // Declaração e criação
7         Stack<String> pilha = new Stack<>();
8
9         // Adicionando elementos
10        pilha.push("Ana");
11        pilha.push("Carlos");
12        pilha.push("Maria");
13
14        // Exibindo pilha
15        System.out.println("Pilha: " + pilha);
16
17        // Consultando o topo
18        System.out.println("Topo: " + pilha.peek());
19
20        // Removendo elemento do topo
21        pilha.pop();
22
23        // Exibindo após remoção
24        System.out.println("Pilha atualizada: " + pilha);
25    }
26 }
```

Capítulo 22 Exercícios

Exercício 22.1

Cadastro de Cliente - Crie um programa em Java que solicite os seguintes dados do cliente:

- ✿ Nome
- ✿ Idade
- ✿ Salário mensal
- ✿ Valor do empréstimo desejado

Ao final, exiba todas as informações digitadas em uma caixa de mensagem. Use JOptionPane para receber os dados.

Exercício 22.2

Análise de Crédito Simples - Desenvolva um programa em Java para realizar uma análise simples de crédito. O programa deve solicitar os seguintes dados do cliente: **(i) Nome; (ii) Salário mensal; (iii) Valor da parcela desejada.**

Considere a seguinte regra de análise. Se o valor da parcela for menor ou igual a 30% do salário, o crédito pode ser analisado. Caso contrário, o crédito deve ser negado. Ao final, o sistema deve exibir: (i) Nome do cliente; (ii) Salário mensal; (iii) Valor da parcela desejada; (iv) Resultado da análise.

Objetivo: aplicar cálculo percentual, estrutura condicional composta e uso de componentes da interface gráfica Swing. Use elementos da interface gráfica Swing (por exemplo: showInputDialog, showConfirmDialog, showOptionDialog)

Exercício 22.3

Classificação de risco - Crie um programa que leia: (i) Nome do cliente; (ii) Salário mensal; (iii) Score de crédito e classifique o cliente da seguinte forma: Score acima de 800: risco baixo, Score entre 600 e 800: risco médio, e Score abaixo de 600: risco alto. Exiba a classificação em uma mensagem.

Objetivo: praticar decisões encadeadas.

Exercício 22.4

Sistema simples de aprovação de Crédito - Crie um programa que receba: (i) Nome, (ii) Idade, (iii) Salário, (iv) Valor do empréstimo desejado, (v) Número de parcelas. Após isso ele vai calcular automaticamente o valor da parcela, usando a conta

$$\text{parcela} = \frac{\text{valor do empréstimo}}{\text{número de parcelas}}$$

Depois o algoritmo deve aplicar as regras para aprovação de crédito:

- ✿ Idade mínima de 18 anos
- ✿ Salário maior que 0
- ✿ Parcela menor ou igual a 30% do salário

Se todas as condições forem atendidas, exiba **“Crédito aprovado”**. Caso contrário, exiba **“Crédito negado”**.

Objetivo: Integrar entrada de dados, processamento matemático e múltiplas condições lógicas.